

Universidad Nacional de Cuyo
Facultad de Ingeniería

Proyecto Final de Estudios

Ingeniería en Mecatrónica

Integración de Sistema de Visión Artificial y Planificación Dinámica de Trayectorias

Continuación y evolución del sistema robótico de Agüero y Lezcano

Alumno: Matías Exequiel Molina
Director: Eric Sanchez

Mendoza, Argentina
2026

Índice general

1. Introducción	3
1.1. Contexto del Proyecto	3
1.2. Evolución, Diagnóstico y Reingeniería del Sistema	3
1.3. Justificación y Objetivos	5
1.3.1. Objetivo General	5
1.3.2. Objetivos Específicos	5
2. Estado del Arte y Antecedentes	7
2.1. El Proyecto de Agüero y Lezcano	7
2.2. Manipuladores con mecanismo de cuatro barras en impresión 3D	8
2.3. Integración de micro-ROS en Sistemas Embebidos	8
2.4. Planificación de Trayectorias Dinámicas en Pick-and-Place	9
3. Arquitectura del Sistema	10
3.1. Evolución de la Arquitectura de Software y ROS (Antes y Después) .	10
3.2. Arquitectura Interna y Flujo de Datos	11
3.2.1. Nivel de Control Embebido (Firmware)	12
3.2.2. Puente de Comunicación (micro-ROS)	20
3.2.3. Nivel de Supervisión y Orquestación (ROS 2)	20
3.2.4. Diagramas de Secuencia e Interacción de la Celda	24
3.3. Protocolos de Interacción Operativos	25
3.3.1. Comando Punto a Punto Articular (P2P)	25
3.3.2. Secuencia de Referencia (Homing)	26
3.3.3. Control del Efecto Final (Electroimán / Solenoide)	26
3.3.4. Parada por Software y Alcance de Seguridad (E-Stop)	26
3.3.5. Planificación Cartesiana y Trayectorias Quínticas (ctraj)	27
4. Desarrollos y Mejoras Implementadas	28
4.1. Evolución del Modelado Cinemático	28
4.1.1. Análisis del Modelo Anterior y Justificación del Cambio	28
4.1.2. Nueva Aproximación: Método Geométrico Directo	28
4.1.3. Cinemática Directa (FK)	29
4.1.4. Cinemática Inversa (IK)	29
4.2. Reingeniería de acoples mecánicos (Unión por chavetero)	31
4.3. Optimización de la integridad de señal en buses I2C	32
4.4. Implementación del Nodo de Visión Artificial y Calibración	32

4.4.1.	Calibración Dinámica y Umbrales en Espacio HSV:	33
4.4.2.	Extracción del Centroide y Publicación en ROS 2	33
4.4.3.	Mapeo Métrico por Ajuste Polinómico (Píxel a Milímetro): . .	34
4.4.4.	Test Experimental y Análisis de Dispersión del Sistema de Visión	35
4.4.5.	Clasificación Geométrica de Objetos Invariante a la Rotación .	36
4.5.	Diseño e Implementación de la Interfaz Gráfica de Usuario (GUI) . .	37
4.6.	Evolución hacia el Modo Dinámico Continuo en 1 Paso (V2)	42
4.6.1.	Estimación de Velocidad por Compuertas Espaciales en Visión	42
4.6.2.	Modelo de Sincronización Temporal e Intercepción en Y_{pick} . .	43
4.6.3.	Unificación y Criterio del Filtro de Factibilidad Cinemática . .	43
4.6.4.	Control en Bucle Cerrado de la Posición de Guardia (± 2.0 mm)	44
4.6.5.	Secuencia de Despegue Vertical Puro (<i>Vertical Lift</i>) y Retorno Rápido	44
4.6.6.	Comparativa de Métodos de Estimación de Velocidad (V1 vs. V2)	45
4.7.	Diseño y Fabricación de la Cinta Transportadora	46
5.	Resultados y Validación Cuantitativa	47
5.1.	Evaluación y Cumplimiento de Objetivos	47
5.2.	Metodología de Validación y Banco de Ensayos	47
5.3.	Caso de Uso 1: Pick-and-Place Estático con Retroalimentación Visual	48
5.3.1.	Descripción del Escenario	48
5.3.2.	Resultados Cuantitativos y Registro Experimental	48
5.4.	Caso de Uso 2: Intercepción Dinámica Síncrona a Velocidad Nominal	49
5.4.1.	Descripción del Escenario y Registro Experimental	49
5.4.2.	Resultados Cuantitativos y Análisis Estadístico	49
5.5.	Evaluación de Límites Operativos: Variación Dinámica de Velocidad .	50
5.6.	Registro Audiovisual y Demostraciones	51
6.	Conclusiones, Limitaciones y Trabajos Futuros	53
6.1.	Conclusiones	53
6.2.	Limitaciones del Sistema	53
6.3.	Trabajos Futuros	54
6.4.	Guía de Reproducibilidad y Entorno Técnico	54
6.5.	Declaración sobre el Uso de Inteligencia Artificial	55

1. Introducción

1.1. Contexto del Proyecto

El presente Proyecto Final de Estudios (PFE) constituye una evolución técnica y funcional del manipulador serial de 3 GDL con mecanismo de paralelogramo articulado de cuatro barras para operaciones tipo “pick-and-place”, desarrollado previamente por Emanuel Agüero y Agustín Lezcano [1]. El sistema original ya integraba ROS 2 como estándar de comunicación, una base sólida que este proyecto retoma y potencia para transformar el prototipo en una celda de manufactura más estructurada, escalable y alineada con las buenas prácticas de la ingeniería de software y control embebido.

1.2. Evolución, Diagnóstico y Reingeniería del Sistema

La motivación principal para dar continuidad al proyecto desarrollado por Agüero y Lezcano surgió de la necesidad de evolucionar el prototipo original. El sistema heredado ya integraba clasificación por visión artificial para la manipulación de elementos comerciales, tales como rodamientos, tuercas, tornillos y llaves fijas. Sin embargo, la operación original se limitaba a un enfoque completamente estático, recogiendo las piezas dispuestas directamente sobre una superficie fija de trabajo. El objetivo de este nuevo proyecto fue dar un salto cualitativo hacia el realismo de una celda de manufactura en operación continua, para lo cual se diseñó e incorporó una cinta transportadora física. Esto permitió migrar hacia un sistema dinámico capaz de interceptar los objetos en movimiento basándose íntegramente en la retroalimentación de la visión artificial.

Al recibir la maqueta física y el código fuente, la etapa inicial de relevamiento presentó una alta complejidad técnica, demandando una revisión exhaustiva fundamentada en los conceptos teóricos de robótica y sistemas embebidos de tiempo real [2, 3]. El firmware original operaba sobre un archivo monolítico (`main.c`) con más de 1000 líneas de código. A nivel de software de alto nivel, si bien el nodo de supervisión en ROS 2 era funcional para sus desarrolladores originales, su operación carecía de una interfaz visual orientada al usuario. Debido a la complejidad en la estructura de los comandos y a la curva de aprendizaje inicial del ecosistema ROS 2, durante las primeras pruebas prácticas solo se logró aislar y replicar exitosamente el accionamiento básico del solenoide del efector final. Ante este escenario, y con el

propósito de establecer una base sólida, comprensible y escalable para el desarrollo futuro y para futuros alumnos, se decidió realizar una reestructuración profunda.

El primer paso consistió en modularizar el firmware del microcontrolador y desarrollar desde cero una nueva Interfaz Gráfica de Usuario (GUI) interactiva. Esta interfaz se dotó de controles deslizantes para el posicionamiento angular directo de cada motor y comandos discretos para la secuencia de *homing* y la sujeción magnética. El proceso de integración comenzó vinculando la telemetría de los sensores magnéticos de posición hacia la interfaz, lo que permitió observar en tiempo real el comportamiento cinemático al mover el brazo manualmente. Tras verificar esta lectura, se logró la correcta ejecución del *homing* y el solenoide. Posteriormente, se implementó un nuevo tópico dedicado a la recepción de comandos articulares punto a punto (P2P), generando trayectorias con perfiles de velocidad trapezoidal. Esto marcó el primer hito de control autónomo del manipulador.

Sin embargo, durante la fase de validación de movimientos surgieron fallas intermitentes. Los motores Q2 (hombro) y Q3 (muñeca) presentaban vibraciones severas, y en ocasiones, el motor principal de elevación (Q2) perdía la tracción, girando en vacío al recibir una consigna. Para aislar el problema, se procedió a desmontar el brazo y enviar comandos a los motores desacoplados de la estructura, comprobando que el software y la electrónica operaban correctamente. La inspección física reveló dos problemas fundamentales: a nivel eléctrico, un cable de fase del motor Q3 presentaba cortes internos; y a nivel mecánico, existían deficiencias en los acoples de transmisión. Se comprobó que el engranaje impreso del motor Q3 había cedido ante el momento torsor, destruyendo el alojamiento de los prisioneros de fijación, mientras que el motor Q2 había estado operando únicamente por fricción.

Para solucionar esta falla estructural, se llevó a cabo una reingeniería de los acoples. Utilizando un torno y fresas para metal, se mecanizaron los ejes cilíndricos de los motores paso a paso para incorporar un sistema de unión por chaveta. Simultáneamente, se rediseñaron e imprimieron en 3D nuevos engranajes que incluyeran el chavetero, manteniendo la fijación por prisionero original pero trasladando el esfuerzo de torsión a la chaveta. Esta intervención mitigó sustancialmente el deslizamiento y las holguras rotacionales.

Una vez resueltas las fallas de transmisión mecánica, un desafío adicional surgió al integrar el control espacial mediante coordenadas cartesianas (X, Y, Z) en la nueva interfaz. Inicialmente, se empleó el modelo cinemático heredado, el cual utilizaba la convención de Denavit-Hartenberg (D-H) e introducía un eslabón y articulación virtual para modelar matemáticamente la restricción de paralelismo del efector impuesta por el mecanismo de cuatro barras. Sin embargo, la contrastación empírica reveló que esta simplificación no reflejaba con suficiente precisión la pose real del efector final, evidenciando discrepancias sensibles especialmente en la coordenada de altura (Z), la cual resultaba ser la variable más crítica y fácil de medir. Los intentos preliminares de mitigar este error mediante factores de corrección empíricos resultaron insatisfactorios, ya que la calibración convergía solo en regiones acotadas del espacio de trabajo y divergía significativamente en otras.

Tras una revisión de la literatura técnica [4], se decidió descartar el modelo D-H virtualizado y adoptar una aproximación analítica basada en un método geométrico y trigonométrico directo, adaptado específicamente para la morfología de robots con

mecanismo de paralelogramo articulado de cuatro barras. La implementación de este nuevo modelo de cinemática en el software resolvió las incongruencias espaciales sistemáticas. Si bien el sistema conserva un error de posicionamiento residual marginal, se determinó que este es atribuible a las tolerancias de fabricación, holguras de rodamientos e imperfecciones elásticas de las piezas impresas en 3D, resultando un desvío admisible para las exigencias de la tarea estipulada.

Finalmente, con la mecánica y la cinemática estabilizadas, se intentó implementar el seguimiento de trayectorias cartesianas continuas. Se configuró la GUI para enviar tramas de posición a intervalos de 10 ms, buscando la velocidad y reacción necesarias para una futura intercepción dinámica de objetos en la cinta. Sin embargo, el movimiento resultante del brazo era errático y presentaba fuertes vibraciones. Un análisis del flujo de datos determinó que la alta tasa de envío generaba un cuello de botella en el canal de comunicación serial entre el orquestador en la PC (ROS 2) y el microcontrolador (micro-ROS y FreeRTOS), provocando latencias y pérdida de paquetes. Como solución arquitectónica para asegurar un perfil de velocidad continuo y determinista, se tomó la decisión de suprimir el cálculo dinámico continuo del planificador en ROS 2 y migrar la totalidad del cálculo analítico de trayectorias al nivel embebido, aprovechando la Unidad de Punto Flotante (FPU) del STM32.

1.3. Justificación y Objetivos

La selección de este PFE se fundamenta en su capacidad para integrar de manera armónica las diversas áreas de la Mecatrónica. La intervención principal se ha centrado en una reingeniería integral: se ha migrado el control hacia una gestión distribuida mediante **micro-ROS** sobre el **STM32F446RE**, aplicando principios de código limpio y modular. Asimismo, se optimizó la supervisión mediante una nueva interfaz de control (GUI) intuitiva y se corrigieron deficiencias en la transmisión mecánica y la integridad de señales del bus I2C. Este trabajo reafirma la importancia de la mejora continua sobre plataformas experimentales existentes, consolidando el desarrollo inicial como un banco de ensayos funcional y versátil.

1.3.1. Objetivo General

Desarrollar e integrar el sistema de procesamiento de visión artificial y planificación dinámica de trayectorias para una celda de clasificación industrial a escala, utilizando como base el brazo robótico de 3 GDL existente y una cinta transportadora independiente, logrando la detección, seguimiento y manipulación de objetos en movimiento dentro del periodo operativo estipulado.

1.3.2. Objetivos Específicos

- **Relevamiento y Diagnóstico:** Relevar la arquitectura de control, cinemática y estado actual del sistema robótico existente basándose en el informe técnico de Agüero y Lezcano.

- **Definición de Módulos:** Determinar formalmente qué componentes del firmware y hardware existentes se reutilizan de manera directa y cuáles requieren modificaciones para la nueva aplicación.
- **Procesamiento Digital de Imágenes:** Programar un sistema de visión artificial basado en OpenCV (operando en espacio de color HSV) para segmentar, clasificar objetos en categorías básicas y estimar su posición instantánea (X, Y) sobre la banda de transporte.
- **Desarrollo del Planificador Dinámico:** Diseñar e implementar un algoritmo en el controlador STM32 Nucleo capaz de estimar el punto de intercepción futuro en el espacio de trabajo, sincronizando el efector magnético con la velocidad de la cinta transportadora.
- **Validación del Prototipo:** Evaluar el desempeño global del sistema mediante métricas experimentales de tasa de éxito, repetibilidad y límites operativos de velocidad.
- **Supervisión Remota (Opcional):** Configurar de manera secundaria un nodo de comunicación vía ESP32 para el envío de alertas o visualización de estados en un panel local en Node-RED, supeditado al correcto funcionamiento del lazo principal de pick-and-place.

2. Estado del Arte y Antecedentes

2.1. El Proyecto de Agüero y Lezcano

Este PFE se basa en el desarrollo previo realizado por Agüero y Lezcano, quienes implementaron un manipulador serial de 3 GDL con paralelogramo articulado de cuatro barras con visión artificial utilizando YOLOv11n y comunicación vía ROS 2. Como se observa en la Figura 2.1, su trabajo sentó las bases cinemáticas y la primera aproximación a la visión artificial en la plataforma, proporcionando una estructura impresa en 3D que sirve como chasis funcional.

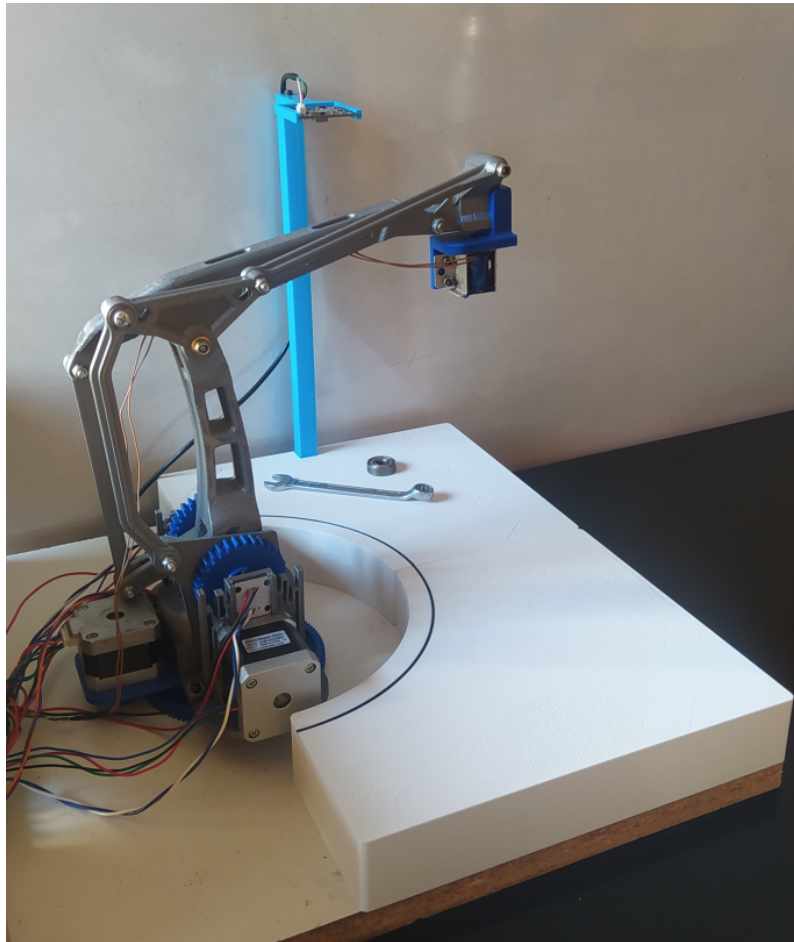


Figura 2.1: PFE de Agüero y Lezcano: Manipulador serial de 3 GDL con mecanismo de cuatro barras y visión artificial.

2.2. Manipuladores con mecanismo de cuatro barras en impresión 3D

En el ámbito de la robótica de código abierto, proyectos como *EezyBotArm* han democratizado el acceso a la manipulación robótica (ver Figura 2.2). La tendencia actual en la comunidad académica es la migración de actuadores servomotores estándar hacia motores paso a paso con encoders absolutos (como los AS5600 integrados), permitiendo una precisión posicional superior en equipos de laboratorio. La integración de estas mejoras, junto con algoritmos de control de tiempo real, sitúa a este proyecto en el estado del arte de las celdas robotizadas a escala.



Figura 2.2: Manipulador serial de 3 GDL con mecanismo de paralelogramo de 4 barras.

2.3. Integración de micro-ROS en Sistemas Embebidos

La evolución desde ROS 1 hacia ROS 2 ha introducido la capacidad de integrar nodos nativos directamente en microcontroladores mediante micro-ROS. Esta tecnología permite a los microcontroladores (como la familia STM32) participar como entidades de primera clase en el grafo de ROS, garantizando determinismo y reduciendo la latencia de capas intermedias. Su aplicación en sistemas robóticos distribuidos es un pilar fundamental en arquitecturas modernas de manufactura flexible.

2.4. Planificación de Trayectorias Dinámicas en Pick-and-Place

Los sistemas tradicionales de manipulación operan de manera estática: el objeto debe detenerse para ser sujetado. Sin embargo, la industria actual requiere la intercepción dinámica. Los algoritmos modernos combinan la estimación de velocidad del objeto (mediante visión artificial o encoders en las cintas) con la generación de trayectorias polinómicas (ej. quinticas) resueltas en tiempo real, permitiendo sincronizar la velocidad del efector final con la de la cinta transportadora en el momento exacto del agarre.

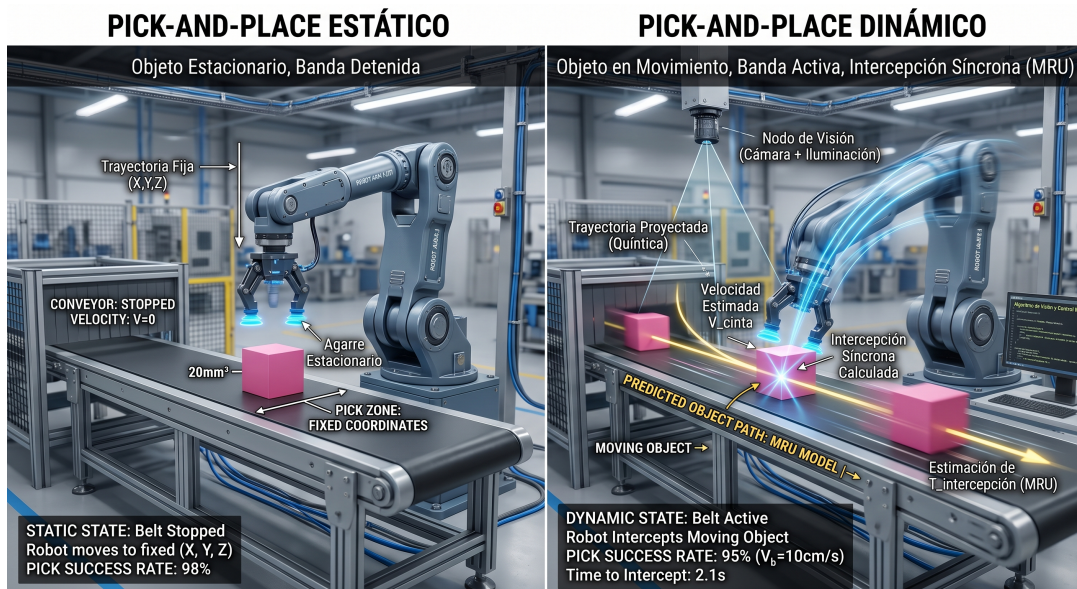


Figura 2.3: Comparativa entre Pick-and-Place estático y dinámico.

3. Arquitectura del Sistema

3.1. Evolución de la Arquitectura de Software y ROS (Antes y Después)

En el nivel de supervisión y alto nivel (ROS 2), la reingeniería abarcó la totalidad de los nodos de control para asegurar un enlace estructurado y de baja latencia con el microcontrolador:

- **Nodo de Visión Artificial:** Mientras que la versión previa dependía de esquemas fragmentados, en este trabajo se desarrolló un nodo de visión propio y modular basado en sockets asíncronos (`VisionClient` y `VisionClientNode`), lo que permite una captura y conversión continua de coordenadas métricas de los objetos detectados.
- **Supresión del Planificador Continuo en ROS:** El nodo planificador continuo que anteriormente operaba en el espacio de ROS 2 fue suprimido de la computadora para las fases dinámicas, reubicando la generación analítica de la curva quántica directamente en el firmware del STM32. Esto redujo drásticamente el tráfico serial y evitó las discontinuidades en los perfiles de velocidad.
- **Interfaz de Usuario (GUI):** La interfaz original presentaba restricciones operativas. Se programó un módulo gráfico interactivo completamente desde cero (`RobotUINode` y vistas en PyQt5) dotado de validación de límites articulares y cartesianos, comandos de homing, cinemática inversa y gestión centralizada de la parada por software (**E-Stop**).
- **Orquestador Central (`MinimalPublisher` / `publisher.py`):** El núcleo de coordinación fue modificado para actuar como compuerta de arbitraje y puente de hardware. Administra la máquina de estados del manipulador (`RobotState`) y la sincronización automática de secuencias operativas multi-segmento ($H \rightarrow A \rightarrow B \rightarrow H$) en conjunto con el servidor de visión.

En la Tabla 3.1 se resume la evolución de las características principales de la arquitectura del sistema.

Dimensión	Versión Anterior	Versión Actual
Organización	Monolítica (<code>main.c</code>)	Modular por componentes
Trayectorias	Procesadas en PC	Planificadas en FPU del STM32
Control	Acoplado a ROS	Distribuido (Tareas FreeRTOS)

Tabla 3.1: Comparativa técnica del firmware.

La migración hacia una arquitectura distribuida permitió la separación de los subsistemas en módulos independientes (*kinematics*, *trajectory_planner*, *microros_callbacks*), facilitando el mantenimiento y la escalabilidad del sistema.

3.2. Arquitectura Interna y Flujo de Datos

La arquitectura de control y comunicación de la celda de manufactura se estructuró en tres niveles de abstracción: el nivel de supervisión y toma de decisiones en la PC host (ROS 2), el puente de comunicaciones de baja latencia (micro-ROS) y el nivel determinista de control embebido en tiempo real. Esta separación garantiza la regularidad en la inyección de señales físicas, un ancho de banda estable para la telemetría y un procesamiento computacional intensivo desacoplado del control directo del hardware.

El subsistema embebido implementado sobre la placa NUCLEO-F446RE desacopla la recepción asíncrona de comandos de alto nivel (vía micro-ROS sobre UART DMA) de la generación determinista de perfiles quínticos y el lazo de control articular ejecutado a 100 Hz bajo FreeRTOS. En la Figura 3.1 se describe el mapa conceptual global de interacción y acoplamiento funcional entre estos módulos.

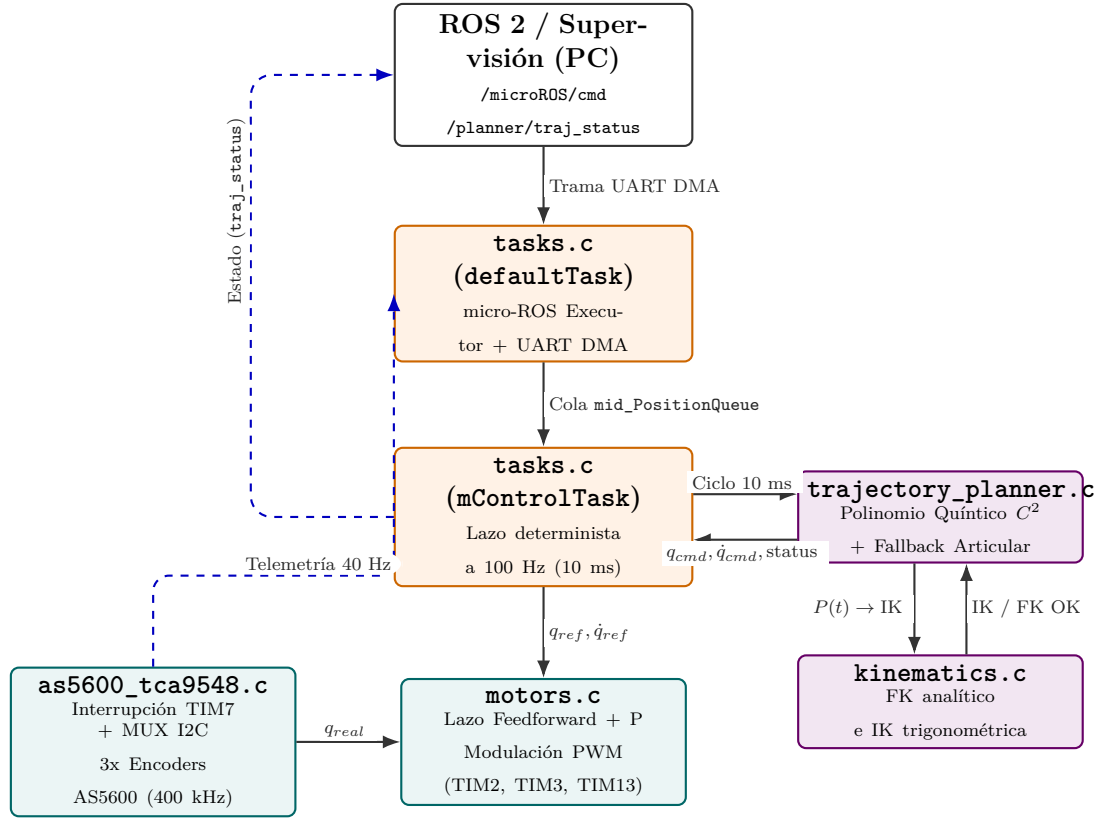


Figura 3.1: Mapa de interacción y acoplamiento funcional de los módulos del firmware.

3.2.1. Nivel de Control Embebido (Firmware)

Para garantizar la ejecución en tiempo real, el firmware del microcontrolador STM32F446RE fue estructurado sobre el sistema operativo FreeRTOS, configurado con un *tick* base de 1 ms y una asignación de memoria dinámica (Heap) de 46 KB. El núcleo de movimiento del robot opera dentro de una tarea dedicada (**mControlTask**) con alta prioridad y una frecuencia de actualización periódica de 100 Hz (10 ms).

La delegación de la planificación de trayectorias al nivel embebido requirió el diseño de algoritmos de control que consideraran los límites dinámicos de los motores paso a paso:

- **Desacople de Hardware:** Las señales PWM son generadas por los temporizadores TIM2, TIM3 y TIM13, mitigando fluctuaciones temporales provocadas por tareas de software. La lectura de los sensores absolutos AS5600 se estructuró mediante el temporizador TIM7 y un multiplexor I2C, capturando la posición articular de manera periódica.
- **Planificador en Dos Fases:** El algoritmo genera un perfil polinómico de quinto grado (C^2). Se limita la velocidad máxima exigida a los actuadores al 85 % de su capacidad física admisible (establecida en 1000 Hz). Si la duración demandada exige violar este límite, el firmware recalcula la duración ajustándola a la cota mínima calculada en el microcontrolador ($t_{\min_fw} \geq 0.60$ s).

- **Asentamiento Terminal:** Al finalizar la curva quíntica, se aplica un control local ($K_{p_traj} = 3.5$) con tolerancia angular (0.45°). Para evitar demoras provocadas por la inercia o la fricción estática de los eslabones impresos en 3D, se configuró una guarda de tiempo límite (*timeout*) de 0.80 s, tras el cual se corta la excitación PWM y se concluye la trayectoria informando el estado a la máquina superior.

Descripción Técnica de los Diagramas de Firmware:

- **Bloques Principales (Figura 3.2):** Ilustra el desacoplo temporal del sistema embebido. La tarea `defaultTask` gestiona la recepción asíncrona de comandos por micro-ROS y deposita las consignas en `mid_PositionQueue`. La tarea de alta prioridad `mControlTask` corre a 100 Hz, consumiendo la cola para alimentar el lazo de control y la generación de pulsos PWM hacia los drivers A4988, mientras los encoders AS5600 proveen retroalimentación angular periódica.
- **Detalle 1 (Figura 3.3):** Detalla el flujo de interrupciones sobre la UART2 con DMA circular. El ejecutor `rc1c` procesa las suscripciones de trayectoria cartesiana, consignas punto a punto articulares, control del solenoide y parada por software. Un temporizador interno a 40 Hz publica la telemetría articular sin saturar el bus serial.
- **Detalle 2 (Figura 3.4):** Muestra el flujo de cálculo en punto flotante (FPU). Al recibir una meta, se valida la cinemática inversa y se ejecuta un perfil quíntico C^2 . Si el sistema entra en la fase terminal, transiciona a un lazo proporcional con ganancia $K_{p_traj} = 3.5$ y una ventana de asentamiento de 0.80 s para absorber la fricción mecánica residual.
- **Detalle 3 (Figura 3.5):** Modela la capa de abstracción de hardware (HAL). Los temporizadores TIM2, TIM3 y TIM13 modulan la frecuencia en hercios para los motores paso a paso. En paralelo, el temporizador TIM7 dispara la máquina de estados no bloqueante por I2C que interroga cíclicamente al multiplexor TCA9548A y a los tres sensores magnéticos AS5600.

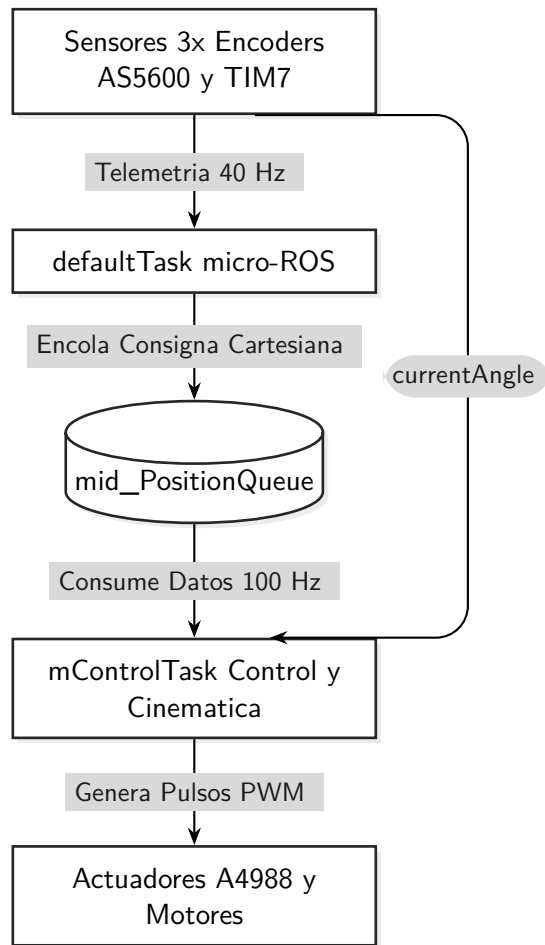


Figura 3.2: Diagrama de bloques Firmware: Bloques Principales de Alto Nivel.

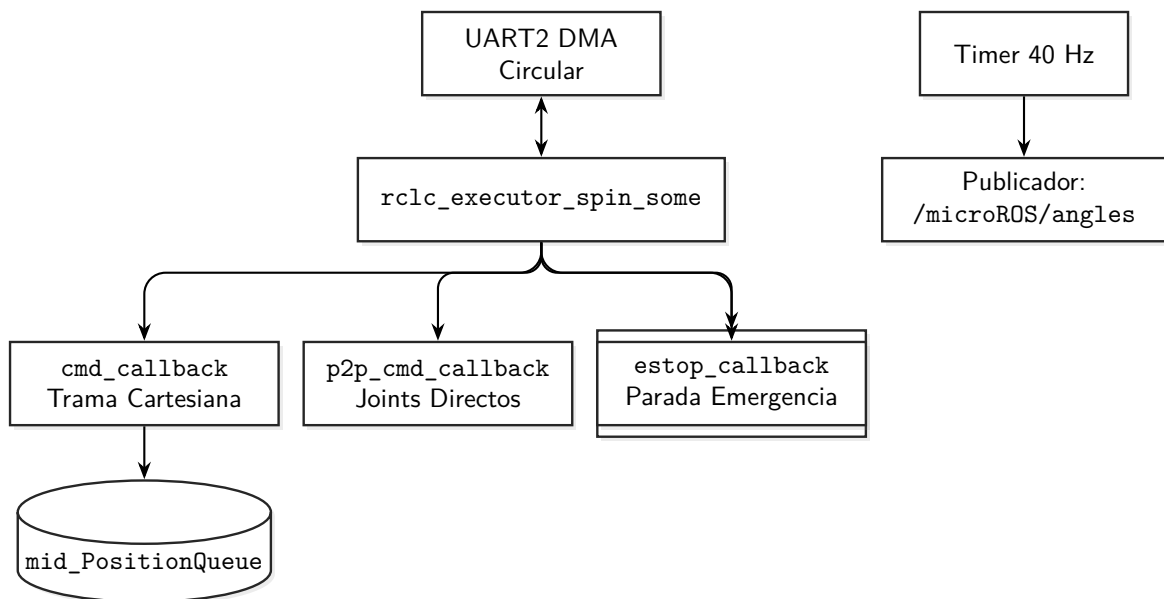


Figura 3.3: Diagrama de bloques Firmware Detalle 1: defaultTask y micro-ROS.

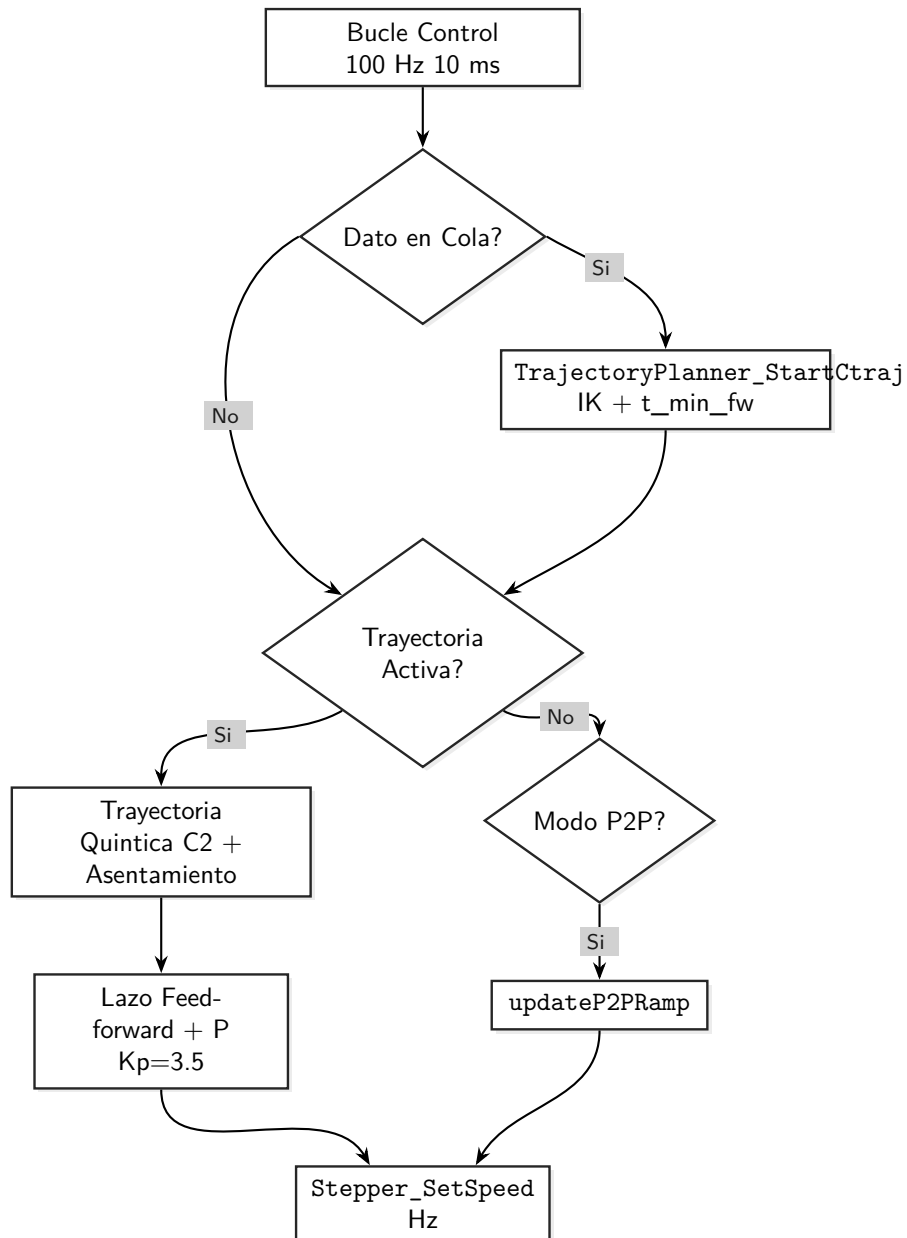


Figura 3.4: Diagrama de bloques Firmware Detalle 2: mControlTask y Planificador Quintico.

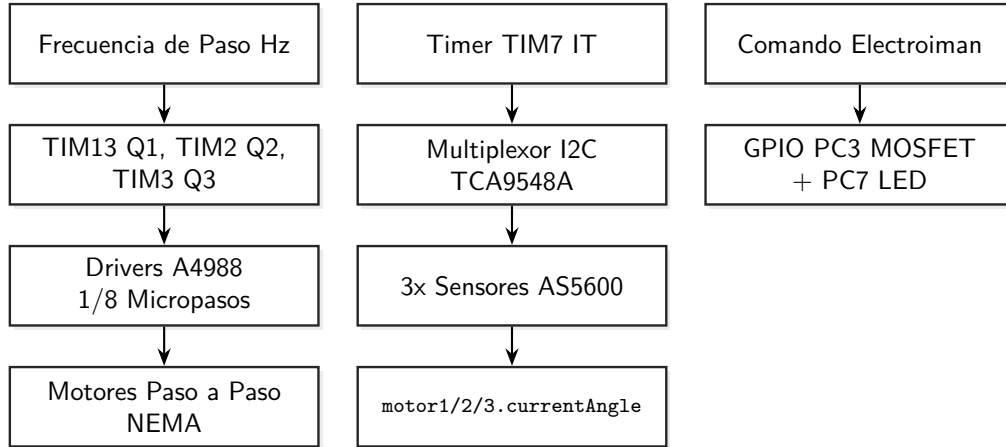


Figura 3.5: Diagrama de bloques Firmware Detalle 3: Periféricos y Hardware Físico.

Parámetro / Macro	Valor	Módulo	Justificación y Criterio
configTOTAL_HEAP_SIZE	46 KB	FreeRTOS.h	Memoria dinámica para stacks de tareas y buffers de micro-ROS.
CONTROL_LOOP_PERIOD	100 Hz (10 ms)	tasks.c	Periodo de cómputo del planificador cuántico y control.
timer_period	40 Hz (25 ms)	tasks.c	Telemetría angular sin saturar canal UART/DMA.
Bus I2C	400 kHz	main.c	Muestreo de 3 encoders AS5600 vía TIM7 sin bloqueo.
MAX_V_HZ	1000 Hz	motors.h	Límite a 1/8 micropaso para evitar pérdidas de paso.
max_freq_safe	850 Hz (85 %)	traj_planner.c	Margen del 15 % para control proporcional sin saturación.
t_min_fw	0.60 s	traj_planner.c	Cota temporal inferior del generador cuántico embebido.
Kp_traj	3.50	motors.c	Ganancia proporcional de seguimiento y asentamiento.
ANGLE_TOLERANCE	0.25° / 0.45°	main.h	Ventana de convergencia para corte de excitación PWM.
Timeout Asentamiento	0.80 s	traj_planner.c	Límite temporal para evitar demoras por fricción residual.

Tabla 3.2: Parámetros operativos y criterios de diseño del Firmware (STM32 / FreeRTOS).

Estructura del Lazo Articular: Control Feedforward + P

A diferencia de un esquema PID clásico puro donde la señal depende enteramente de la acumulación histórica del error, el seguimiento dinámico de las trayectorias cartesianas se rige por un esquema de prealimentación de velocidad (*feedforward*) acoplado a una realimentación proporcional (P) de posición [5].

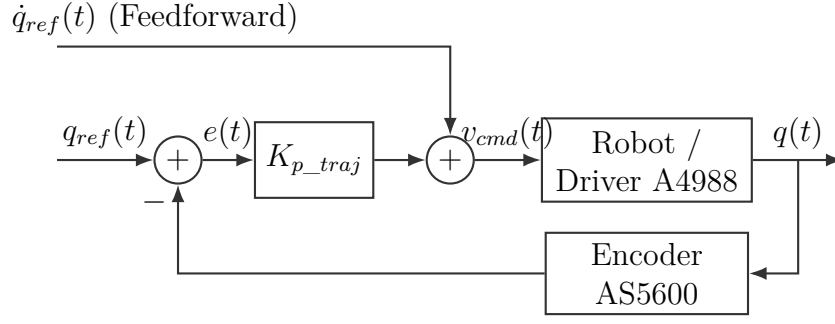


Figura 3.6: Diagrama en bloques del lazo de control con prealimentación de velocidad (feedforward) y realimentación proporcional de posición.

1. Modo Trayectoria Continua Quintica (TRAJ): Velocidad Feedforward a 100 Hz En este modo, la referencia es un **vector dinámico instantáneo** $\dot{q}_{ref}(t)$ (q_{d_ref} en $[\circ/s]$) evaluado cada $\Delta t = 10$ ms mediante la derivada analítica discreta de las consignas quinticas:

$$\dot{q}_{ref}[i] = \frac{q_{current}[i] - q_{last}[i]}{\Delta t} \quad [\circ/s] \quad (3.1)$$

Esta señal entra directamente como prealimentación (*feedforward*) al lazo implementado en `motors.c`:

$$v_{cmd}(t) = \dot{q}_{ref}(t) + K_{p_traj} \cdot (q_{ref}(t) - q_{real}(t)) \quad [\circ/s] \quad (3.2)$$

donde $K_{p_traj} = 3.50$. La velocidad resultante se convierte en pulsos de excitación PWM para los controladores (A4988) considerando la reducción mecánica i y la resolución a 1/8 de micropaso (1600 pulsos/rev, equivalente a `DEG_TO_STEPS = 4.444...` pulsos/ $\circ$):

$$f_{PWM} [Hz] = |v_{cmd}| \cdot \left(\frac{1600 \text{ pulsos}}{360^\circ} \right) \cdot i \quad (3.3)$$

Cotas de saturación:

- **Corte por Velocidad Mínima:** Si $f_{PWM} < 1.0$ Hz (`TRAJ_CUTOFF_FREQ_HZ`), se desactiva la salida para evitar zumbidos residuales en los bobinados.
- **Límite Dinámico Superior:** $f_{PWM} \leq 1000$ Hz (`MAX_V_HZ`). El planificador restringe la velocidad pico de la curva al 85 % (850 Hz, `TRAJ_MAX_SAFE_FREQ_HZ`) para reservar un 15 % de margen al término proporcional.

2. Modo Punto a Punto (P2P): Velocidades de Régimen y Rampas En traslados manuales o rutinas de origen (*homing*) gestionadas por temporizadores de hardware:

- **Velocidad Nominal por Defecto:** `SPEED_P2P_DEFAULT_HZ = 350.0` Hz.
- **Velocidad Base de Arranque:** `MIN_SPEED_HZ = 150.0` Hz.
- **Sincronización Multieje:** `startSynchronizedP2PMovement()` calcula la velocidad de régimen $v_{scaled} \in [280, 1000]$ Hz para cada eje escalándola al tiempo del motor con mayor desplazamiento angular.

Responsabilidad Específica de los Módulos del Firmware

`main.c` — Configuración del Microcontrolador y Recursos Base

- **Inicialización del Reloj:** Núcleo STM32F446RE a 180 MHz vía oscilador HSI acoplado al PLL ($M = 8$, $N = 180$, $P = 2$).
- **Mapeo de Temporizadores PWM (Shield A4988):**
 - TIM13 (CH1) → Motor 1 (Cintura / Q1) — 16 bits (ARR máx: 65535).
 - TIM2 (CH2) → Motor 2 (Hombro / Q2) — 32 bits (ARR máx: 4294967295).
 - TIM3 (CH2) → Motor 3 (Muñeca / Q3) — 16 bits (ARR máx: 65535).
- **Prevención de Overflow en ARR:** Como $ARR = (f_{timer}/f_{PWM}) - 1$, frecuencias menores a 2 Hz desbordarían registros de 16 bits. `Stepper_SetSpeed()` satura a 65535 y corta el pulso si $f < 2$ Hz.
- **Recursos FreeRTOS:** Instancia `mid_PositionQueue` con capacidad para 4 consignas `CARTESIAN_CMD_t`.

`tasks.c` — Concurrencia FreeRTOS y Transporte micro-ROS

- **defaultTask (Prioridad Normal — Stack 8 KB):** Atiende micro-ROS vía UART2 DMA (115200 bps). Encola consignas cartesianas desde `/microROS/cmd`, procesa la parada por software en `/microROS/emergency_stop` y publica ángulos a 40 Hz en `/microROS/angles`.
- **mControlTask (Alta Prioridad — 100 Hz, Stack 4 KB):** Ejecutada periódicamente cada 10 ms con `vTaskDelayUntil()`. Desencola consignas de `mid_PositionQueue`, evalúa `TrajectoryPlanner_Update()`, modula el lazo Feedforward + P hacia `motors.c` y publica el estado en `/planner/traj_status`.

trajectory_planner.c — Generador Quintico y Fallback Articular Genera perfiles suaves con velocidad y aceleración nulas en extremos:

$$s(\tau) = 10\tau^3 - 15\tau^4 + 6\tau^5, \quad \tau = \frac{t}{T} \in [0, 1] \quad (3.4)$$

1. **Fase de Arranque (StartCtraj):** Valida la cinemática inversa (IK) del destino y registra la pose articular inicial $\mathbf{q}_{start}$. Calcula el tiempo mínimo dinámico local evaluando la derivada pico (1.875):

$$t_{min_fw} = \max \left(0.60 \text{ s}, \max_{i \in \{1,2,3\}} \frac{1.875 \cdot \Delta q_i \cdot \text{DEG_TO_STEPS} \cdot i_{red}[i]}{850 \text{ Hz}} \right) \quad (3.5)$$

Nótese que este valor representa la cota inferior algorítmica del microcontrolador ante desplazamientos mínimos, diferenciándose de la guarda operativa del supervisor (T_{MIN_ROBOT}) requerida para el traslado completo desde la guardia hasta la cinta.

2. **Fase 1 ($\tau < 1.0$ — Interpolación Quíntica y Fallback):** Interpola $P(t) = P_{start} + s(\tau)(P_{target} - P_{start})$. Si la recta cartesiana atraviesa una singularidad o excede el espacio de trabajo, `Kinematics_IK()` falla y el firmware conmuta de forma continua al espacio articular:

$$q_{cmd}[i](t) = q_{start}[i] + s(\tau) \cdot (q_{target}[i] - q_{start}[i]) \quad (3.6)$$

garantizando arribo a destino sin discontinuidades bruscas de velocidad ni interrupción del ciclo.

3. **Fase 2 ($\tau \geq 1.0$ — Asentamiento Terminal):** Mantiene q_{target} con feed-forward nulo. Supervisa que $|q_{target}[i] - q_{real}[i]| \leq 0.45^\circ$. Si converge retorna SUCCESS (2); si expiran 0.80 s (`TRAJ_SETTLING_TIMEOUT_S`) concluye por TIMEOUT (3).

`kinematics.c`, `motors.c` y `as5600_tca9548.c`

- **`kinematics.c`:** Geometría de cuatro barras: $L_1 = 77.0$ mm, $L_2 = 135.0$ mm, $L_3 = 147.0$ mm, $O_r = 25.49$ mm, $O_z = 40.87$ mm. Límites articulares de seguridad: $Q_1 \in [-90^\circ, 90^\circ]$, $Q_2 \in [40^\circ, 157^\circ]$, $Q_3 \in [-75^\circ, 25^\circ]$.
- **`motors.c`:** Ley $v_{cmd} = \dot{q}_{ref} + 3.5(q_{ref} - q_{real})$ traducida a pulsos PWM al 50 % de ciclo de trabajo.
- **`as5600_tca9548.c`:** Muestreo I2C Fast-Mode (400 kHz) por TIM7 en Round-Robin ($1 \rightarrow 2 \rightarrow 3$) con filtro de rechazo $> 180^\circ$.

Ciclo de Ejecución y Códigos de Estado

Para asegurar la sincronización bidireccional entre el firmware embebido y los nodos de supervisión en ROS 2 (o interfaz gráfica de usuario), el planificador publica periódicamente su estado a través del tópico `/planner/traj_status` empleando los códigos normalizados detallados en la Tabla 3.3.

Código	Estado del Planificador	Criterio de Disparo en el Firmware
1	<code>TRAJ_STATUS_RUNNING</code>	Maniobra en curso: interpolación quíntica ($\tau < 1.0$) o asentamiento ($\tau \geq 1.0$).
2	<code>TRAJ_STATUS_SUCCESS</code>	Conclusión exitosa con convergencia estática en los tres ejes ($\leq 0.45^\circ$).
3	<code>TRAJ_STATUS_TIMEOUT</code>	Fin por expiración de guarda de asentamiento ($t_{settling} \geq 0.80$ s).
-1	<code>TRAJ_STATUS_ERROR_IK_START</code>	Rechazo inicial en <code>StartCtraj()</code> : punto destino inalcanzable.
-2	<code>TRAJ_STATUS_ERROR_IK_PATH</code>	Estado reservado ante fallas dinámicas críticas irreversibles durante interpolación.
-3	<code>TRAJ_STATUS_ABORT_ESTOP</code>	Cancelación forzada por parada de emergencia en <code>/microROS/emergency_stop</code> .

Tabla 3.3: Códigos de sincronización publicados en `/planner/traj_status`.

3.2.2. Puente de Comunicación (micro-ROS)

El enlace entre el procesamiento de alto nivel y el control físico se realiza mediante el estándar DDS-XRCE. Una tarea dedicada (`defaultTask`), operando con prioridad normal y un *stack* de 8 KB, administra el *executor* de micro-ROS.

Esta capa intermedia gestiona la comunicación serial a través de una interfaz UART configurada con DMA circular, asegurando que la recepción asíncrona de datos no bloquee el control periódico de los motores. Para prevenir la sobrecarga del bus de datos y mantener la regularidad del sistema, el envío de telemetría articular hacia el orquestador se reguló mediante un temporizador de software interno que publica los mensajes a una frecuencia de 40 Hz (cada 25 ms). Al recibir una trama cartesiana desde el supervisor, el *callback* la deriva inmediatamente a una cola de mensajes no bloqueante (`mid_PositionQueue`) para su consumo por la tarea de control.

En la Figura 3.7 se presenta la estructura global distribuida que interconecta la capa de supervisión en ROS 2 con el hardware embebido a través del agente micro-ROS.

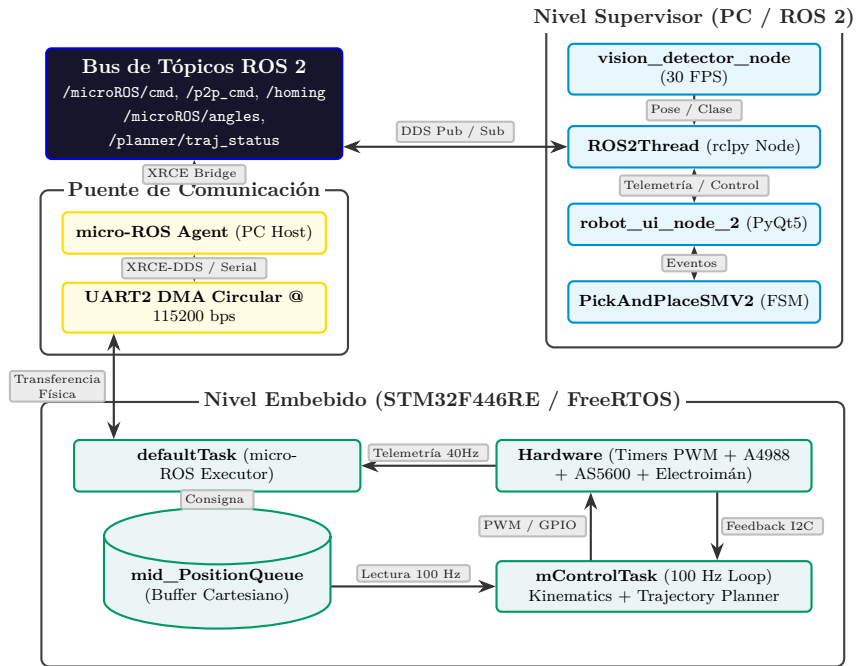


Figura 3.7: Diagrama Global: Arquitectura Distribuida del Sistema.

3.2.3. Nivel de Supervisión y Orquestación (ROS 2)

El nivel superior opera en la computadora principal y concentra el procesamiento de visión artificial, el cálculo analítico de intercepción y la máquina de estados operativa. Al delegar la ejecución temporal de los motores al microcontrolador, el orquestador garantiza una toma de decisiones de alto nivel basada en condiciones de validación geométrica y temporal.

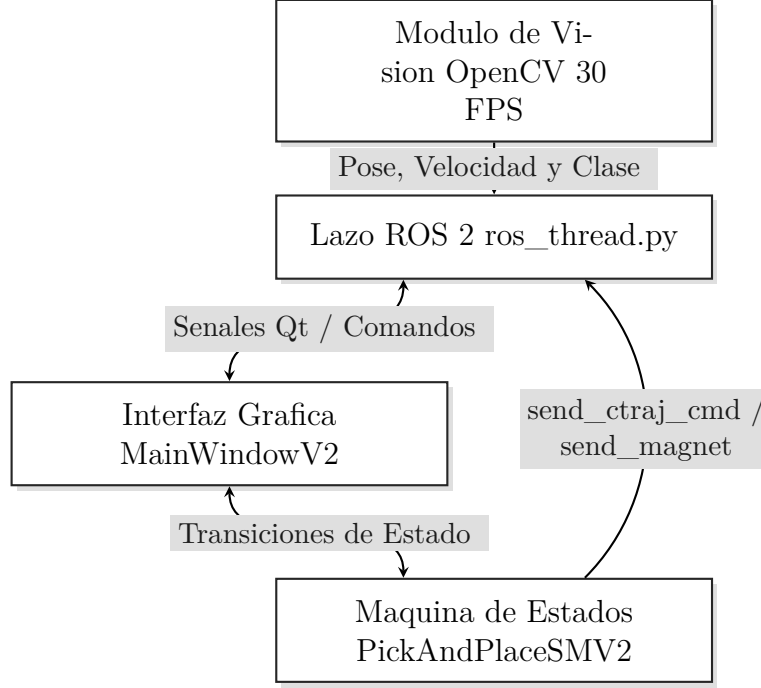


Figura 3.8: Diagrama de arquitectura y flujo de control del sistema.

Módulos Centrales de la Capa de Supervisión

vision_detector_node.py — Percepción Cenital, Velocidad y Clasificación Este nodo opera a 30 FPS en espacio de color HSV, asumiendo las siguientes responsabilidades analíticas:

- **Adquisición y Calibración Métrica Cuadrática:** Captura video a 640×480 a 30 FPS. Tras segmentar el objeto en espacio HSV y filtrar ruido con clausuras y aperturas morfológicas, extrae el centroide (c_x, c_y) mediante momentos geométricos. La coordenada longitudinal Y sobre la cinta se obtiene mediante el polinomio calibrado:

$$Y_{robot} [\text{mm}] = a \cdot c_x^2 + b \cdot c_x + c \quad (3.7)$$

donde $a = 7.7532 \times 10^{-5}$, $b = 0.5865$ y $c = -185.7812$. El eje transversal de la cinta se encuentra posicionado en $X_{\text{FIXED}} = 215.0$ mm y la altura de contacto en $Z_{\text{FIXED}} = 78.0$ mm.

- **Estimación Lineal de Velocidad por Compuertas Fijas:** Configura dos barreras espaciales transversales a la cinta: $Y_{G1} = -160.0$ mm y $Y_{G2} = -90.0$ mm ($\Delta Y = 70.0$ mm). Al cruzar la pieza Y_{G1} , registra la marca de tiempo de alta resolución t_1 (`time.perf_counter()`). Al cruzar Y_{G2} , calcula la velocidad escalar:

$$v_{cinta} = \frac{Y_{actual} - Y_{G1}}{t_2 - t_1} \quad [\text{mm/s}] \quad (3.8)$$

publicando el vector $(v_{cinta}, Y_{actual}, flag_{medido})$ en el tópic `/detected_object_dynamic_pose`.

- **Clasificación Morfológica Invariante a la Rotación (Cubo vs. Cono):** Evalúa la relación de cajas y densidad sobre el contorno en el cerrojo óptico ($-40.0 \leq Y \leq -20.0$ mm):

$$Ratio_{cajas} = \frac{\text{Área Bounding Box Rotado}}{\text{Área Bounding Box Recto}}, \quad Extent_{rot} = \frac{\text{Área Contorno}}{\text{Área Bounding Box Rotado}} \quad (3.9)$$

Si la pieza rota ($Ratio_{cajas} < 0.85$) o su ocupación es densa ($Extent_{rot} > 0.85$), se clasifica como **Cubo** (clase = 1). De lo contrario, se clasifica como **Cono** (clase = 2).

pick_and_place_sm_v2.py — Máquina de Estados y Factibilidad Orquesta las transiciones operativas mediante un autómata finito de estados continuos:

- **Unificación Matemática del Filtro de Factibilidad:** Ante la detección de un objeto a velocidad v_{cinta} , la máquina de estados calcula el tiempo disponible de arribo al punto de agarre ($Y_{target} = 120.0$ mm):

$$t_{disp} = \frac{Y_{target} - y_{actual}}{v_{cinta}} - T_{anticipo} \quad (3.10)$$

donde $T_{anticipo} = 0.15$ s compensa el retardo del solenoide. Simultáneamente, evalúa el tiempo mínimo demandado por el robot (T_{MIN_ROBOT}) a partir de la cinemática inversa destino y la pose articular instantánea:

$$T_{MIN_ROBOT} = \max \left(0.60 \text{ s}, \max_{i \in \{1,2,3\}} \frac{1.875 \cdot |\Delta q_i| \cdot \text{DEG_TO_STEPS} \cdot i_{red}[i]}{850 \text{ Hz}} \right) \quad (3.11)$$

En caso factible ($t_{disp} \geq T_{MIN_ROBOT}$), conmuta al estado de intercepción despachando la consigna con duración t_{disp} . En caso no factible ($t_{disp} < T_{MIN_ROBOT}$), conmuta a aborto preventivo notificando velocidad excesiva en pantalla sin mover el robot.

- **Manejo del Protocolo de Estados de Trayectoria:** Los códigos de retorno del firmware coordinan las transiciones: códigos de falla o parada por software (≤ -1) desenergizan el electroimán y abortan el ciclo; el código de ejecución activa (1) mantiene la espera; y los códigos de éxito o fin de guarda (2 y 3) habilitan el avance hacia el despeque vertical a $Z = 125$ mm, verificación óptica y descarga en contenedores clasificados.

ros_thread.py y robot_ui_node_2.py — Desacoplamiento Gráfico Aloja el ejecutor bloqueante `rclpy.spin()` en un hilo secundario (`QThread`), evitando degradar el renderizado visual de la GUI a 60 FPS. Mapea callbacks de tópicos ROS 2 hacia señales tipificadas de Qt: telemetría articular a 40 Hz (`feedback_received`), eventos de finalización de trayectoria (`planner_status_received`), paquetes de tracking y velocidad (`vision_dynamic_target_received`) y transmisión de fotografías (`vision_image_received`). La interfaz principal integra la visualización de la clasificación en tiempo real, estimación de velocidad, monitor de estados y parada por software.

publisher.py — **Enrutador Puente de Hardware y Arbitraje** Actúa como compuerta de arbitraje para mandos manuales y paradas de emergencia. Intercepta comandos de homing, movimientos articulares P2P y excitación del efector final provenientes de la interfaz gráfica. Si la bandera de seguridad por software **is_estop_active** está enclavada, bloquea de forma inmediata la publicación de tramas hacia micro-ROS, garantizando que el firmware no reciba consignas en estados inseguros.

3.2.4. Diagramas de Secuencia e Interacción de la Celda

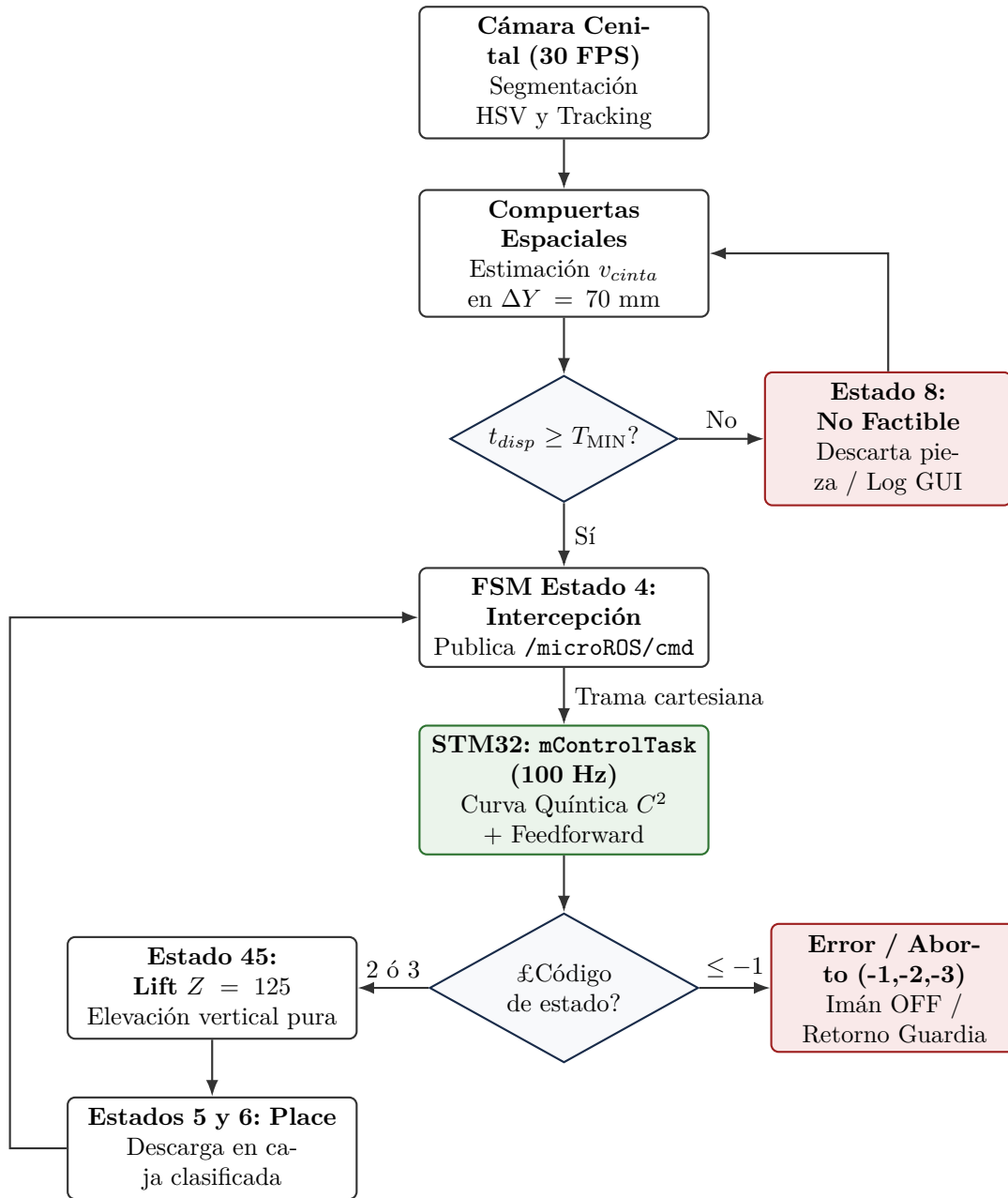


Figura 3.9: Flujograma general de decisión, validación de factibilidad y ejecución de ciclo.

Parámetro / Variable	Valor	Módulo	Justificación y Criterio
Tasa Captura Cénital	30 FPS (33.3 ms)	vision_node.py	Muestreo sin distorsión temporal.
Resolución Video	640 × 480 px	vision_node.py	Balance de resolución y baja latencia.
Compuertas $Y_1 \rightarrow Y_2$	-140 → -70 mm	vision_node.py	Tramo fijo ($\Delta Y = 70$ mm) para medir velocidad.
Bloqueo Óptico	[-40, -20] mm	sm_v2.py	Fijación de clase en centro de lente sin oclusiones.
Extent / Rect Ratio	≥ 0.85	vision_node.py	Discriminación invariante a la orientación.
Meta Intercepción	$Y_{\text{pick}} = 120.0$ mm	sm_v2.py	Posición nominal en espacio de trabajo.
T_MIN_ROBOT	Dinámico / 0.85 s	sm_v2.py	Filtro analítico de descarte cinemático.
T_{anticipo}	0.15 s	sm_v2.py	Compensación de retardo electromagnético.
POS_TOLERANCE	±2.0 mm	sm_v2.py	Verificación en lazo cerrado para reposo en guardia.
Elevación (Z_{lift})	125.0 mm (0.6 s)	sm_v2.py	Despegue en Z pura para evitar corte por fricción.
Failsafe Agarre	$Y > 135.0$ mm	sm_v2.py	Detección de caída y retorno rápido (0.9 s).

Tabla 3.4: Parámetros de orquestación, visión y máquina de estados (ROS 2 / Python).

3.3. Protocolos de Interacción Operativos

La arquitectura distribuida interconecta el nivel de supervisión en ROS 2 (Python / PyQt5) con la capa de control en la STM32F446RE (C / FreeRTOS / micro-ROS). A continuación se detallan los flujos de interacción para las principales funcionalidades operativas de la celda.

3.3.1. Comando Punto a Punto Articular (P2P)

El envío de consignas articulares directas permite posicionar el manipulador especificando los ángulos de cada articulación ($\theta_1, \theta_2, \theta_3$):

- **Capa de Supervisión (GUI):** El usuario ajusta los valores mediante los deslizadores o selectores numéricos en la pestaña de control manual de la interfaz gráfica.
- **Capa de Comunicación (ROS 2):** El método `send_cmd` del hilo de comunicación empaqueta los ángulos en un mensaje del tipo `geometry_msgs/msg/Point` y los publica a través del tópico `/microROS/p2p_cmd`.

- **Capa Embebida (micro-ROS / Firmware):** El ejecutor del microcontrolador detecta el nuevo dato y ejecuta la función `p2p_cmd_callback`. Esta rutina invoca a `startSynchronizedP2PMovement()`, la cual calcula los perfiles de rampa de aceleración y velocidad sincronizada para los tres motores paso a paso.

3.3.2. Secuencia de Referencia (Homing)

La rutina de calibración y posicionamiento inicial al origen del robot se gestiona mediante el siguiente flujo:

- **Capa de Supervisión (GUI):** Al presionar el botón de homing, la interfaz activa la señal correspondiente.
- **Capa de Comunicación (ROS 2):** Se publica un mensaje de tipo booleano ('True') a través del tópico `/microROS/homing`.
- **Capa Embebida (micro-ROS / Firmware):** El callback `homing_callback` recibe la orden y ejecuta la función `doAllHoming()`. Esto desplaza de forma sincronizada cada articulación hacia su ángulo de referencia de homing configurado por hardware.

3.3.3. Control del Efecto Final (Electroimán / Solenoide)

La manipulación y sujeción de piezas metálicas se rige por un control digital de estado:

- **Capa de Supervisión (GUI):** El operador conmuta el estado del imán desde la interfaz, lo cual genera un mensaje de control.
- **Capa de Comunicación (ROS 2):** El nodo orquestador o la UI publican un mensaje tipo `std_msgs/msg/Bool` en el tópico `/microROS/electroiman`.
- **Capa Embebida (micro-ROS / Firmware):** La función `electromagnet_callback` procesa el mensaje y llama a `electromagnetOn()`, la cual conmuta físicamente los pines del GPIO y el LED indicador mediante la HAL de ST.

3.3.4. Parada por Software y Alcance de Seguridad (E-Stop)

El sistema implementa una respuesta prioritaria ante solicitudes de detención generadas desde la interfaz de control:

- **Capa de Supervisión (GUI):** Al pulsar la parada en cualquier pestaña, se emite una orden prioritaria de detención.
- **Capa de Comunicación (ROS 2):** El enrutador puente (`publisher.py`) enclava la bandera `is_estop_active`, bloqueando de inmediato cualquier comando hacia el robot, y difunde un booleano ('True') a través de los tópicos `/microROS/emergency_stop` y `/planner/emergency_stop`.

- **Capa Embebida (micro-ROS / Firmware):** El callback `estop_callback` aborta inmediatamente cualquier trayectoria activa en curso, conmuta el estado de los actuadores a `MOTOR_ERROR` y detiene de forma forzosa la modulación PWM de los temporizadores para suprimir la excitación de los bobinados.

Delimitación del Alcance de Seguridad: Con rigor técnico, debe señalarse explícitamente que este mecanismo constituye una **parada de detención por software** implementada sobre la capa lógica de comunicación. Por tanto, **no debe considerarse como una parada de emergencia de seguridad funcional certificada bajo estándares industriales (tales como ISO 13849 o IEC 62061)**, puesto que depende de la disponibilidad del canal serial UART, la pila DDS de micro-ROS y el hilo supervisor en la PC host, careciendo de un circuito físico redundante y cableado directo de corte de energía (desconexión electromecánica de potencia).

3.3.5. Planificación Cartesiana y Trayectorias Quínticas (`ctray`)

La ejecución de movimientos suaves en el espacio cartesiano utiliza las capacidades de cálculo local del microcontrolador:

- **Capa de Supervisión (GUI):** Se ingresa la meta cartesiana (X, Y, Z) y la duración deseada de la trayectoria, enviando una trama de datos estructurada.
- **Capa de Comunicación (ROS 2):** El método `send_ctray_cmd` publica una estructura de tipo `extra_interfaces/msg/Trama` a través del tópico `/microROS/cmd`.
- **Capa Embebida (micro-ROS / Firmware):** El callback `cmd_callback` recibe la trama y encola los datos cartesianos de manera no bloqueante en `mid_PositionQueue`. La tarea de alta prioridad `mControlTask` evalúa la cola, inicializa el planificador cartesiano mediante `TrajectoryPlanner_StartCtray()`, resuelve la cinemática inversa y ejecuta el control Feedforward + P en tiempo real sobre los motores.

4. Desarrollos y Mejoras Implementadas

4.1. Evolución del Modelado Cinemático

4.1.1. Análisis del Modelo Anterior y Justificación del Cambio

En la versión inicial del proyecto, la cinemática directa se abordó utilizando la convención de Denavit-Hartenberg (D-H) mediante matrices de transformación homogénea. Para la cinemática inversa, se empleó una aproximación geométrica con desacople cinemático que requirió la adición de un eslabón virtual (L_4) y un grado de libertad ficticio (q_4) para tratar al mecanismo como una cadena cinemática abierta.

Si bien este método es funcional, la introducción de estos artificios matemáticos genera un modelo indirecto para un mecanismo con lazo cerrado de cuatro barras, lo que aumenta la complejidad analítica y el riesgo de divergencias en cálculo embebido.

4.1.2. Nueva Aproximación: Método Geométrico Directo

Tras una revisión de la literatura técnica [4], se decidió descartar el modelo D-H virtualizado y adoptar una aproximación analítica basada en un método geométrico y trigonométrico directo, adaptado específicamente para la morfología de robots con mecanismo de paralelogramo articulado de cuatro barras. Este método mapea fielmente la morfología del manipulador a partir de las dimensiones físicas validadas mediante el ensamblaje en SolidWorks, eliminando las articulaciones virtuales y absorbiendo los desfases directamente en el cálculo del radio.

Los parámetros geométricos definitivos establecidos para esta versión son:

- $L_1 = 77.00$ mm (Altura de la base al hombro)
- $L_2 = 135.00$ mm (Longitud del brazo principal)
- $L_3 = 147.00$ mm (Longitud del antebrazo)
- $O_r = 25.49$ mm (Offset radial del efector final)
- $O_z = 40.87$ mm (Offset vertical del efector final)

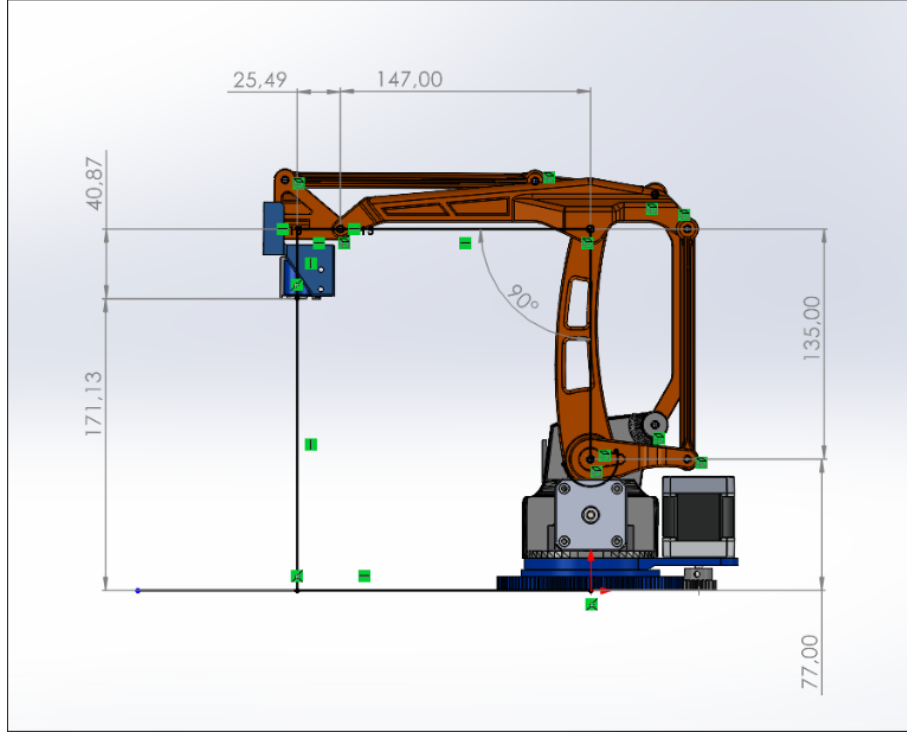


Figura 4.1: Validación de parámetros geométricos y offsets del manipulador en SolidWorks.

4.1.3. Cinemática Directa (FK)

Conocidos los ángulos articulares absolutos Q_1 , Q_2 y Q_3 , la posición cartesiana (X, Y, Z) del efector final se obtiene proyectando los eslabones sobre el plano de trabajo. El radio total R incorpora de manera directa el offset radial:

$$R = L_2 \cos(Q_2) + L_3 \cos(Q_3) + O_r \quad (4.1)$$

A partir de este radio, las coordenadas espaciales resultan en:

$$X = R \cos(Q_1) \quad (4.2)$$

$$Y = R \sin(Q_1) \quad (4.3)$$

$$Z = L_1 + L_2 \sin(Q_2) + L_3 \sin(Q_3) - O_z \quad (4.4)$$

4.1.4. Cinemática Inversa (IK)

Dada una coordenada cartesiana objetivo (X, Y, Z) , el ángulo de la base se determina por proyección directa en el plano horizontal:

$$Q_1 = \text{atan2}(Y, X) \quad (4.5)$$

Para resolver el mecanismo en el plano vertical, se define el desplazamiento hacia el nodo de la muñeca restando los offsets constructivos:

$$r = \sqrt{X^2 + Y^2} - O_r \quad (4.6)$$

$$z' = Z + O_z - L_1 \quad (4.7)$$

La distancia lineal D entre la articulación del hombro y la muñeca resulta en:

$$D = \sqrt{r^2 + (z')^2} \quad (4.8)$$

Aplicando el Teorema del Coseno al triángulo físico formado por los eslabones L_2 , L_3 y la distancia D , se obtienen los ángulos internos β y γ :

$$\beta = \arccos \left(\frac{D^2 + L_2^2 - L_3^2}{2 \cdot D \cdot L_2} \right) \quad (4.9)$$

$$\gamma = \arccos \left(\frac{D^2 + L_3^2 - L_2^2}{2 \cdot D \cdot L_3} \right) \quad (4.10)$$

Conociendo el ángulo de elevación de D respecto a la horizontal, definido como $\alpha = \text{atan2}(z', r)$, los ángulos absolutos de las articulaciones quedan definidos por las siguientes expresiones, garantizando implícitamente la configuración “codo arriba”:

$$Q_2 = \alpha + \beta \quad (4.11)$$

$$Q_3 = \alpha - \gamma \quad (4.12)$$

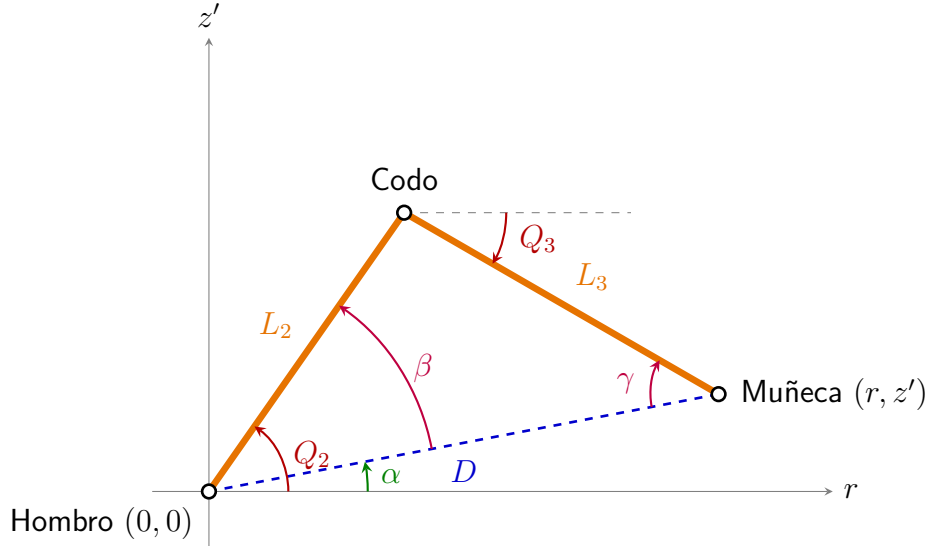


Figura 4.2: Diagrama trigonométrico de la Cinemática Inversa en el plano proyectado $r - z'$, mostrando las relaciones angulares de los eslabones.

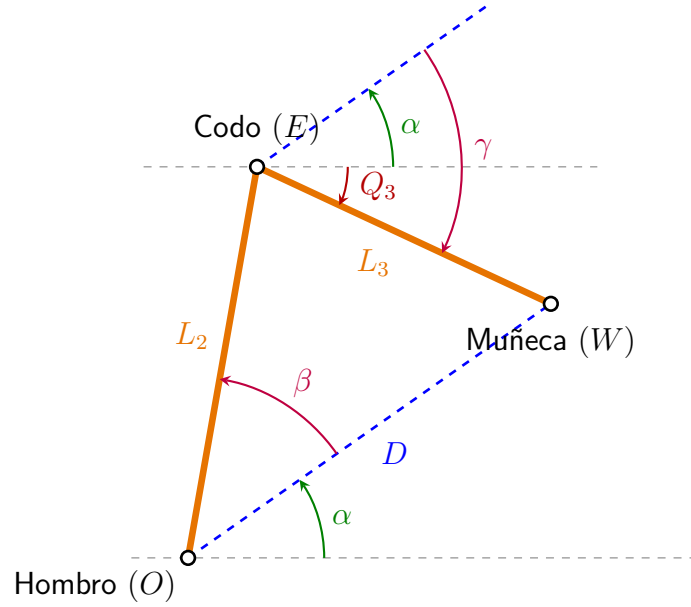
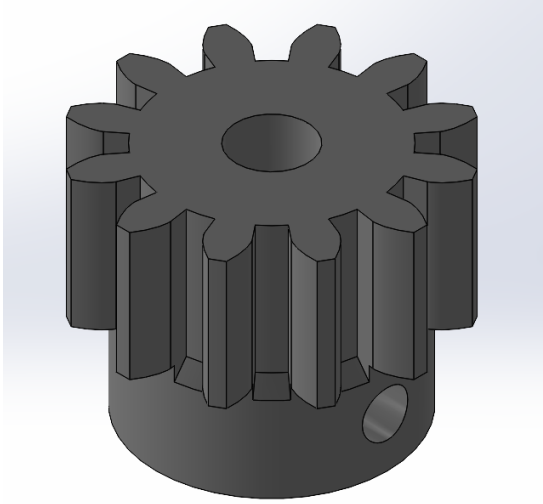


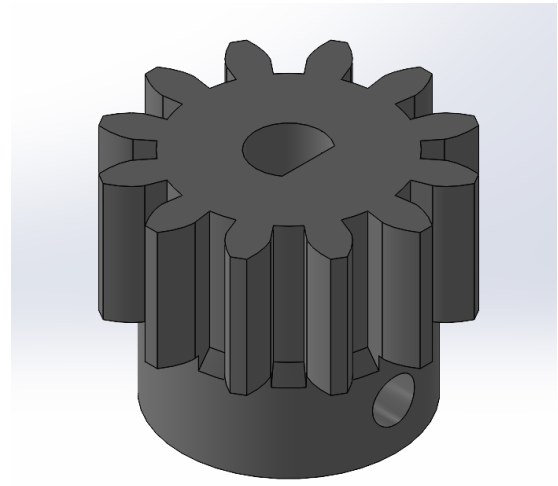
Figura 4.3: Demostración visual del ángulo Q_3 . Al trazar una línea paralela a D desde el Codo, se observa por ángulos alternos internos que la inclinación absoluta del antebrazo es $Q_3 = \alpha - \gamma$.

4.2. Reingeniería de acoples mecánicos (Unión por chavetero)

Con el fin de resolver los problemas de deslizamiento y holgura torsional detectados en los acoples originales del sistema heredado, se llevó a cabo una reingeniería integral de la transmisión mecánica. Esta intervención consistió en el mecanizado de los ejes cilíndricos de los motores paso a paso para incorporar un alojamiento preciso, combinado con el rediseño y fabricación de los engranajes plásticos impresos en 3D para integrar una unión por chaveta. Esta solución proporciona una transmisión de torque rígida y mitiga de forma sustancial las desalineaciones angulares durante la operación continua de los ejes.



(a) Engranaje original con acople por fricción.



(b) Engranaje impreso en 3D con unión por chaveta.

Figura 4.4: Comparativa de la reingeniería mecánica de engranajes.

4.3. Optimización de la integridad de señal en buses I2C

Durante las pruebas iniciales del sistema, se detectaron anomalías e intermitencias en la comunicación con los sensores magnéticos basados en el protocolo I2C. Tras una revisión detallada del hardware, se identificó que el arnés de cables original presentaba una disposición física desfavorable: las líneas de reloj (*SCL*) y de datos (*SDA*) se encontraban trenzadas entre sí. Esta disposición propicia el acoplamiento capacitivo y la diafonía (*cross-talk*) mutua entre señales digitales de alta frecuencia.

Para reducir el ruido electromagnético inducido y asegurar la estabilidad del bus, se rediseñó el tendido separando las líneas de señal y aplicando un esquema de par trenzado de referencia cerrada: se trenzó de manera independiente la línea de reloj con la referencia a tierra (*SCL* con *GND*), y la línea de datos con la alimentación (*SDA* con *VCC*). Esta modificación atenuó notablemente la susceptibilidad al ruido eléctrico generado por la conmutación de los motores paso a paso, logrando una transmisión de datos estable en el rango operativo del manipulador.

4.4. Implementación del Nodo de Visión Artificial y Calibración

Para dotar al sistema de la capacidad de percibir su entorno, se desarrolló un nodo de visión dedicado en ROS 2 (*VisionDetectorNode*) integrado con OpenCV, tomando como base arquitecturas de percepción aplicadas a brazos robóticos industriales [6]. Este subsistema procesa el flujo de video en tiempo real de una cámara cenital conectada mediante un enlace persistente (*symlink*). Dado que las condiciones de iluminación ambiental afectan de forma directa la segmentación cromática,

el desarrollo contempló herramientas de calibración que favorecen la repetibilidad del sistema:

4.4.1. Calibración Dinámica y Umbrales en Espacio HSV:

Mediante una interfaz auxiliar interactiva (`calibrar_hsv.py`) provista de controles deslizantes (*trackbars*), es posible sintonizar de forma dinámica los rangos cromáticos del espacio de color HSV (Tono, Saturación y Valor). Esta transformación desacopla la información de color de la intensidad lumínica ambiental, aislando con alta fidelidad la silueta del objeto de prueba respecto al fondo de la cinta.

Para purificar la máscara binaria resultante y eliminar los falsos positivos derivados del ruido del sensor o de reflejos en la superficie de la banda, se implementa una secuencia estricta de filtros morfológicos en dos etapas utilizando elementos estructurantes (*kernels*):

1. **Operación de Cierre (*Morphological Close*):** Se aplica un kernel rectangular de 11×11 píxeles de manera iterativa (2 pasadas). Esta operación ejecuta una dilatación seguida de una erosión, lo que permite rellenar pequeños huecos, discontinuidades u oclusiones internas dentro de la silueta segmentada del objeto, asegurando un contorno cerrado y sólido.
2. **Operación de Apertura (*Morphological Open*):** A continuación, se aplica un kernel de menor tamaño de 5×5 píxeles. Consiste en una erosión seguida de una dilatación, destinada a suprimir el ruido disperso de fondo (pequeños puntos o motas aisladas que superan el umbral de color pero no pertenecen a una pieza válida).

4.4.2. Extracción del Centroide y Publicación en ROS 2

Una vez aplicada la máscara de color y filtrado el ruido mediante operaciones morfológicas de apertura y cierre, el nodo analiza los contornos presentes en la escena. Al detectar el objeto de mayor área por encima del umbral establecido, se calculan los momentos espaciales de la imagen mediante la función `cv2.moments()`. A partir de los momentos de orden cero y uno, se obtiene la posición del centroide en píxeles (cx, cy) aplicando las expresiones:

$$cx = \frac{M_{10}}{M_{00}}, \quad cy = \frac{M_{01}}{M_{00}} \quad (4.13)$$

donde M_{00} representa el área total del objeto (momento de orden cero), y M_{10} y M_{01} son los momentos espaciales de primer orden respecto a los ejes X e Y , respectivamente.

Esta posición en píxeles es convertida a coordenadas métricas utilizando el modelo polinómico cuadrático calibrado. Finalmente, el nodo empaqueta estas coordenadas espaciales en mensajes estándar de ROS 2 del tipo `geometry_msgs/Point` y los transmite mediante dos publicadores: uno de flujo continuo para el modo estático y otro síncrono para las rutinas dinámicas del manipulador.

4.4.3. Mapeo Métrico por Ajuste Polinómico (Píxel a Milímetro):

Para traducir las coordenadas de imagen desde el sistema de píxeles hacia unidades métricas reales en la base del robot, se implementó una rutina de calibración espacial. Este procedimiento se realiza posicionando el efector final a lo largo de la cinta transportadora mediante la cinemática directa (FK) de la interfaz gráfica en un rango de $[-140, 140]$ mm sobre el eje Y del robot.

Mediante el script `capturar_punto.py`, se obtienen las coordenadas centroidales en píxeles de la cámara (correspondientes a la posición horizontal x en la imagen) asociadas a cada posición real conocida. A partir de estos datos, recopilados en la Tabla 4.1, se emplea el script de automatización `calcular_polinomico.py`. Este programa interactivo procesa los pares empíricos aplicando un modelo de regresión por mínimos cuadrados mediante la función `np.polyfit()` de NumPy, resolviendo un polinomio de segundo orden de la forma $Y_{\text{robot}} = a \cdot X_{\text{px}}^2 + b \cdot X_{\text{px}} + c$. Su ejecución permite calcular con alta precisión los coeficientes empíricos que compensan los errores de curvatura y distorsión óptica de la lente cenital, garantizando linealidad métrica en todo el trayecto útil sobre la banda.

Punto N°	Y_{real} [mm]	Coordenada X [píxeles]
1	-139.4	539.51
2	-119.7	512.61
3	-99.3	478.64
4	-79.4	445.67
5	-60.3	413.91
6	-40.0	383.16
7	-20.1	354.19
8	0.3	318.16
9	19.8	287.23
10	40.1	250.50
11	60.0	217.86
12	79.8	184.59
13	99.1	153.92
14	119.8	122.78
15	139.5	89.47

Tabla 4.1: Registro completo de adquisición de puntos experimentales para la calibración espacial ($N = 15$).

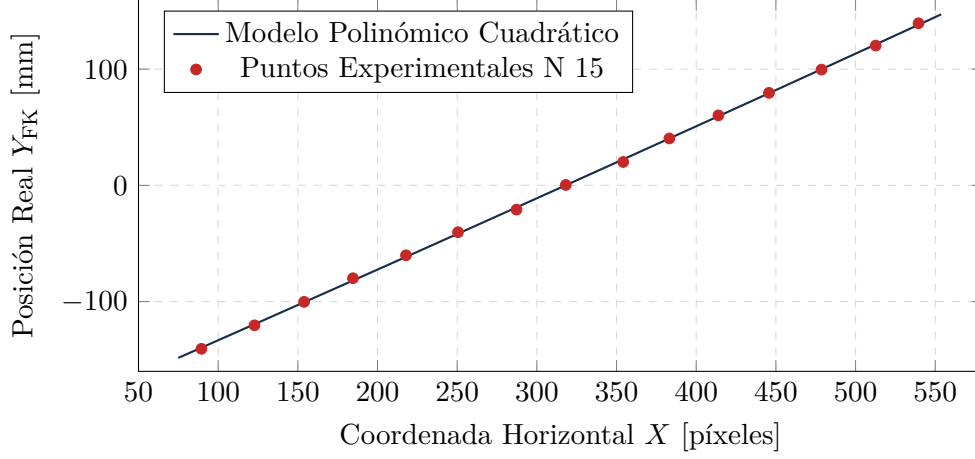


Figura 4.5: Curva de calibración polinómica Y_{FK} frente a X en píxeles, evidenciando la mitigación de la distorsión óptica.

4.4.4. Test Experimental y Análisis de Dispersión del Sistema de Visión

Para cuantificar rigurosamente la precisión métrica del nodo de visión una vez implementado el ajuste polinómico cuadrático, se desarrolló una serie de pruebas sistemáticas directamente sobre la maqueta física de la celda, basándose en la captura de $N = 15$ puntos de control distribuidos longitudinalmente sobre la cinta transportadora.

Metodología del Ensayo: Utilizando la cinemática directa (FK) desde la interfaz gráfica de usuario (GUI), se posicionó de manera estática el efector final con un cubo de referencia patrón en marcas espaciadas cada 20.0 mm de forma sucesiva, abarcando todo el rango operativo de la cinta desde $Y = -140.0$ mm hasta $Y = +140.0$ mm. En cada una de las 15 posiciones discretas, el nodo **VisionDetectorNode** procesó la imagen para estimar la coordenada cartesiana $\hat{Y}_i$ mediante el modelo polinómico, registrando el desvío residual absoluto $e_i = |\hat{Y}_i - Y_{\text{real},i}|$.

Punto N°	Y_{real} [mm]	Y_{vision} [mm]	Error Residual e_i [mm]
1	-139.4	-137.8	1.6
2	-119.7	-118.0	1.7
3	-99.3	-98.3	1.0
4	-79.4	-79.2	0.2
5	-60.3	-60.2	0.1
6	-40.0	-39.6	0.4
7	-20.1	-19.6	0.5
8	0.3	0.5	0.2
9	19.8	20.7	0.9
10	40.1	40.6	0.5
11	60.0	59.3	0.7
12	79.8	79.5	0.3
13	99.1	99.7	0.6
14	119.8	118.7	1.1
15	139.5	140.0	0.5

Tabla 4.2: Muestra representativa del test de dispersión del sistema de visión ($N = 15$ puntos muestreados cada 20 mm).

Análisis Estadístico de Dispersión y Errores: A partir de la serie completa de los 15 puntos registrados experimentalmente, se computaron los indicadores globales de desempeño del algoritmo de percepción:

- **Error Medio (Bias):** +0.41 mm
- **Error Absoluto Medio (MAE):** 0.69 mm
- **Raíz del Error Cuadrático Medio (RMSE):** 0.83 mm
- **Desviación Estándar de Dispersión (σ):** 0.73 mm
- **Error Máximo Residual:** 1.70 mm

Los resultados cuantitativos demuestran que la dispersión del sistema de visión se mantiene dentro de una cota submilimétrica en la región central del campo visual y no supera 1.70 mm en los extremos del área de trabajo. Esta baja dispersión ($\sigma = 0.73$ mm) certifica la alta repetibilidad del nodo de visión y valida la suficiencia del modelo cuadrático para mitigar la distorsión óptica de la lente cenital, asegurando un posicionamiento altamente compatible con la tolerancia de agarre del efector magnético.

4.4.5. Clasificación Geométrica de Objetos Invariante a la Rotación

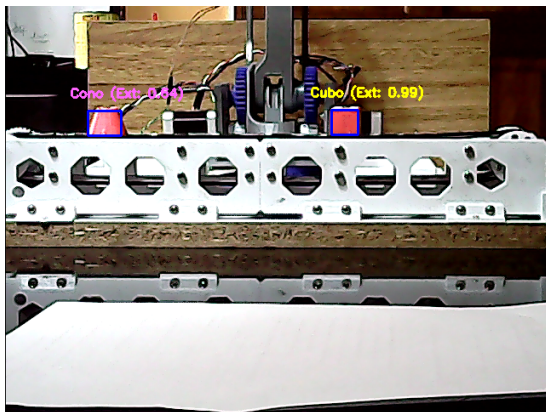
Para habilitar la capacidad de clasificación del sistema, se integró un algoritmo capaz de discriminar entre distintas geometrías de piezas que comparten el mismo color (en este caso, un cubo y un cono truncado de color rosa).

La discriminación basada únicamente en contornos básicos resultaba sensible ante sombras o variaciones de silueta. Inicialmente se evaluó el método del **Extent Rotado** (relación entre el área del objeto y su `minAreaRect`). No obstante, las pruebas experimentales demostraron que cuando el cubo ingresaba orientado en diagonal, el filtrado morfológico redondeaba sus esquinas, reduciendo su coeficiente Extent a valores cercanos a los del cono (≈ 0.81).

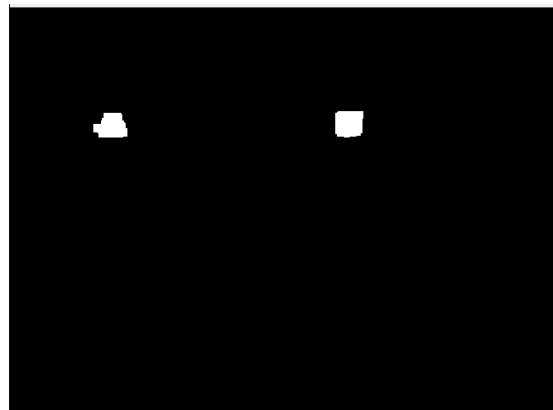
Para resolver este solapamiento y alcanzar una alta tasa de discriminación geométrica independiente de la orientación, se incorporó una segunda métrica basada en la **Relación de Cajas (Rect Ratio)**:

$$\text{Rect Ratio} = \frac{A_{\text{rotada}}}{A_{\text{recta}}} = \frac{\text{Área minAreaRect}}{\text{Área boundingRect}} \quad (4.14)$$

La lógica de decisión combinada opera evaluando la dispersión de área: si el cubo se encuentra rotado, su caja recta encierra una mayor proporción de fondo vacío, produciendo un *Rect Ratio* inferior a 0.85. Si la pieza transita alineada con la cinta, ambas cajas presentan áreas semejantes (*Rect Ratio* ≈ 1.0), instancia en la cual se evalúa su densidad mediante el *Extent* (donde un cubo presenta valores sistemáticamente mayores a 0.85). Este enfoque complementario otorga una clasificación geométrica confiable frente a distintas orientaciones de ingreso de la pieza.



(a) Vista de cámara con caja rotada (`minAreaRect`) y clasificación por Extent.



(b) Máscara HSV segmentada, evidenciando las diferentes geometrías.

Figura 4.6: Clasificación simultánea de geometrías mediante el cálculo del Extent Rotado.

4.5. Diseño e Implementación de la Interfaz Gráfica de Usuario (GUI)

Con el fin de centralizar la supervisión y operación de la celda robótica, se desarrolló una interfaz gráfica de usuario desde cero utilizando el framework **PyQt5**, estructurada en un entorno modular de cinco pestañas principales adaptadas a las distintas fases de experimentación y control. Las Pestañas 2 y 3 están destinadas a la

configuración interna de parámetros y telemetría articular, enfocando la operación de la celda en las Pestañas 1, 4 y 5.

- **Pestaña 1 (Control Manual y P2P):** Diseñada para la operación directa y el posicionamiento manual del manipulador (ver Figura 4.7). Incorpora deslizadores y selectores numéricos con validación de límites articulares para el envío de consignas punto a punto ejecutadas mediante temporizadores en el firmware. Incluye además un cuadro de entrada para coordenadas cartesianas (X, Y, Z) validadas mediante cinemática inversa, interruptores para el accionamiento del solenoide, secuencia de *homing* y botón de parada por software.

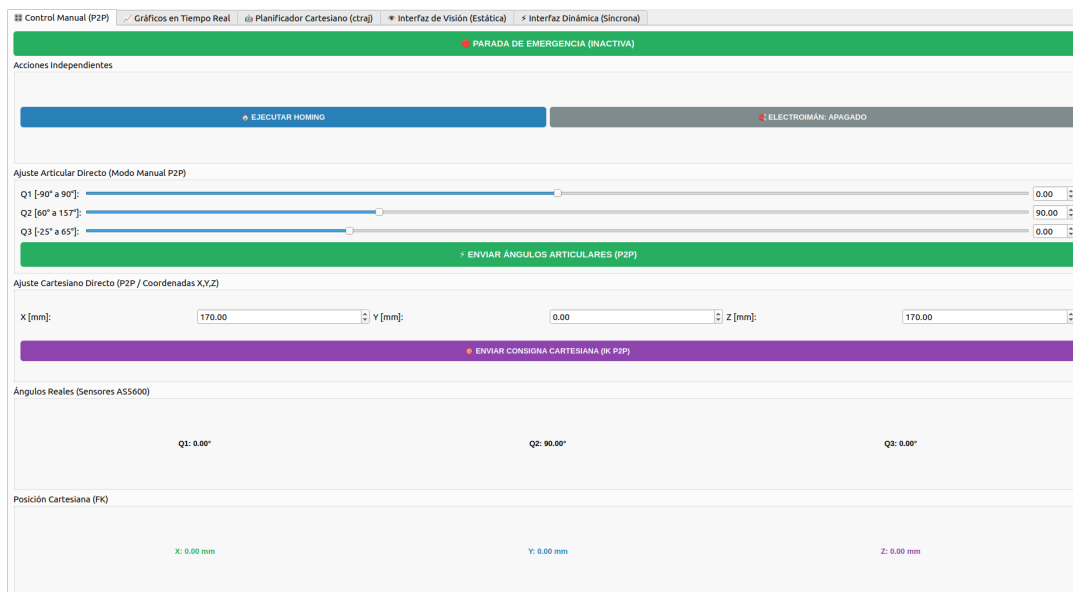


Figura 4.7: Vista general de la Pestaña 1: Control manual, sliders articulares y panel de consignas cartesianas.

- **Pestaña 4 (Pick Estático):** Destinada a la validación de la celda de clasificación con piezas en reposo (ver Figura 4.8). El módulo de visión artificial detecta un cubo de referencia de 20 mm de lado ($20 \times 20 \times 20$ mm) de color rosa y transmite un vector de posición cartesiano (x, y, z) hacia el planificador embebido. Las coordenadas nominales efectivas se establecieron en la cota $Z = 78.0$ mm y posición longitudinal de cinta $X = 215.0$ mm.

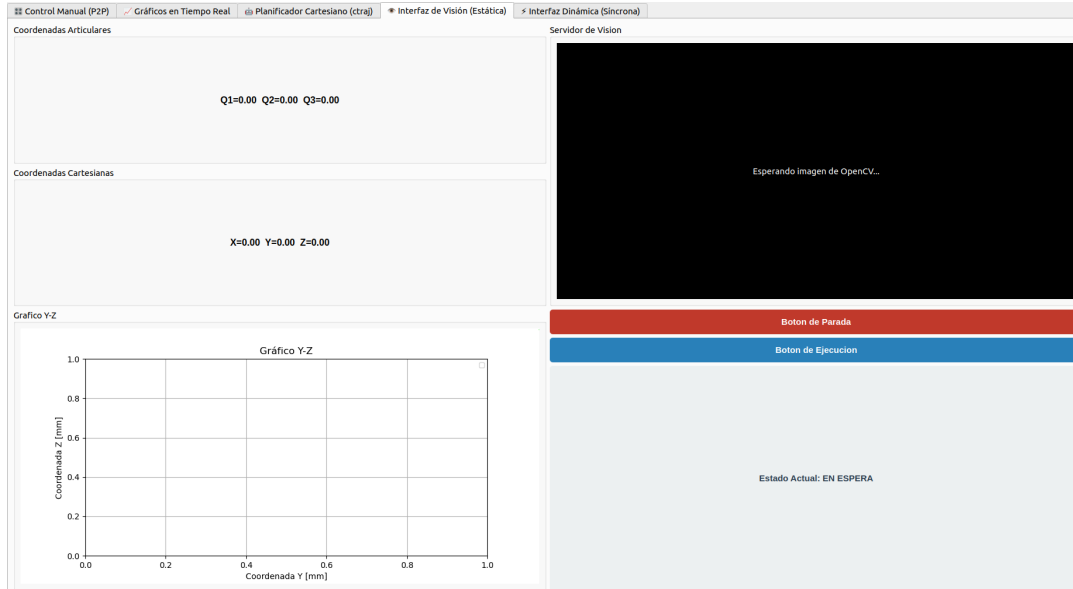


Figura 4.8: Vista de la Pestaña 4: Ensayo de clasificación Pick-and-Place estático con retroalimentación visual.

- **Pestaña 5 (Pick Dinámico y Síncrono V1):** Implementó la primera aproximación operativa para la manipulación de piezas en movimiento sobre la cinta transportadora (ver Figura 4.10). La arquitectura de software se estructuró sobre una máquina de estados determinista que dividió el ciclo en las siguientes etapas operativas:

1. **Calibración Cinemática de la Cinta:** Para estimar la velocidad lineal constante del objeto, se implementó un algoritmo de medición basado en compuertas de tiempo virtuales sobre el flujo de video. Al activar el comando de calibración desde la interfaz, el nodo de visión define dos fronteras espaciales a lo largo del eje longitudinal, separadas por una distancia física conocida de $\Delta Y = 290$ mm. El sistema monitorea el centroide de la pieza; al cruzar la primera compuerta se registra una estampa de tiempo inicial (t_1). Al intersectar la segunda compuerta, se captura el tiempo final (t_2), obteniendo el diferencial temporal $\Delta t = t_2 - t_1$. Finalmente, la velocidad escalar se calcula como $V = \Delta Y / \Delta t$, valor que es transmitido y almacenado globalmente en la máquina de estados para alimentar el modelo predictivo.
2. **Posicionamiento en Guardia y Lazo de Tolerancia:** Previo a la intercepción, el brazo manipulador era comandado hacia una posición de guardia estratégica (e.g., $X = 215, Y = -75, Z = 100$ mm). El algoritmo validaba el arribo del efector final evaluando el error euclídeo en los tres ejes, requiriendo que la posición real se encontrara dentro de una estricta esfera de tolerancia de ± 2.0 mm. Una vez superada esta condición, se energizaba el electroimán de forma anticipada.
3. **Cálculo Predictivo MRU e Intercepción Síncrona:** Con el manipulador en guardia, el sistema evaluaba continuamente la coordenada

longitudinal entrante del objeto (Y_{actual}). En lugar de un seguimiento reactivo ciego, el algoritmo predecía el punto temporal exacto de captura asumiendo un Movimiento Rectilíneo Uniforme (MRU):

$$t_{objetivo} = \left(\frac{Y_{pick} - Y_{actual}}{V} \right) - T_{anticipo} \quad (4.15)$$

donde Y_{pick} (120 mm) representa la coordenada longitudinal fija de intercepción, y $T_{anticipo}$ (0.15 s) compensa el retardo de saturación magnética del solenoide. Siempre que el $t_{objetivo}$ respetara el límite de aceleración física del robot ($t \geq 0.85$ s), se generaba una trayectoria polinómica quíntica que sincronizaba temporal y espacialmente el contacto entre el cabezal y la pieza.

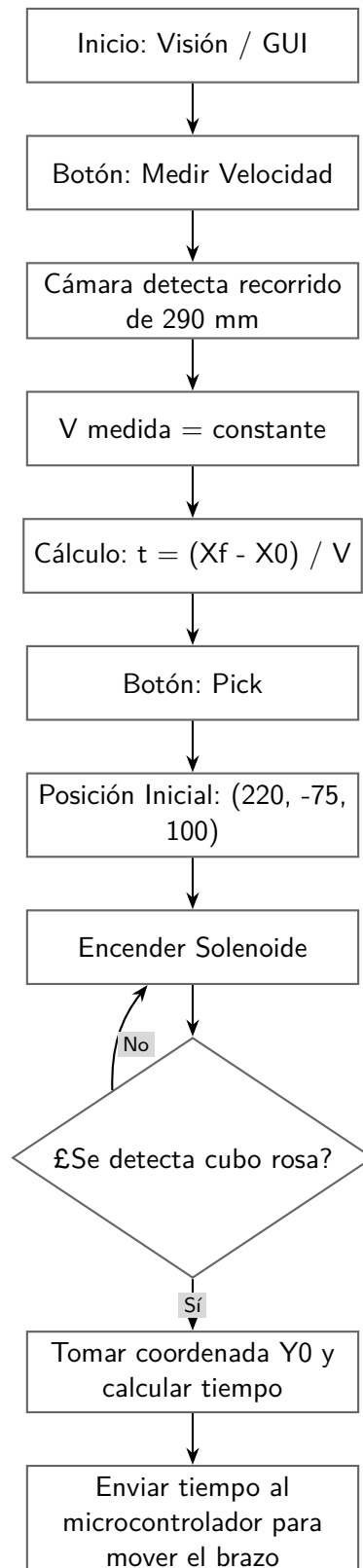


Figura 4.9: Diagrama de flujo del proceso de visión y control.

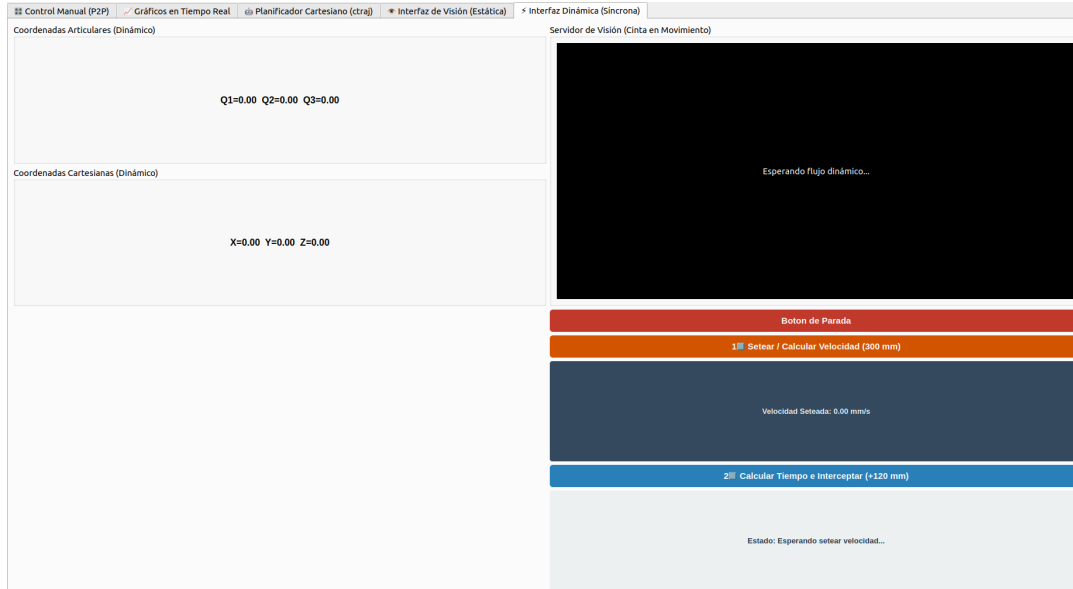


Figura 4.10: Vista de la Pestaña 5: Ensayo de clasificación Pick-and-Place Dinámico.

4.6. Evolución hacia el Modo Dinámico Continuo en 1 Paso (V2)

Para superar la dependencia del operador y optimizar los tiempos de ciclo, se llevó a cabo una reingeniería completa del lazo dinámico, implementada en una arquitectura modular desacoplada (`robot_ui_node_2` y máquina de estados `PickAndPlaceSMV2`). Esta versión V2 sustituyó el esquema de dos etapas por un algoritmo continuo que estima la velocidad de cada pieza y ejecuta la trayectoria de intercepción de forma automática.

4.6.1. Estimación de Velocidad por Compuertas Espaciales en Visión

En los ensayos preliminares de la versión V1, calcular la velocidad en el hilo de la interfaz mediante la diferencia temporal de mensajes ROS 2 introducía desvíos notables (del orden del 20–25 %) debido a la latencia acumulada en los buffers de captura de video y al retardo de refresco gráfico.

Para mitigar las fluctuaciones temporales (*jitter*) y desacoplar la estimación de la carga de la interfaz, el cálculo de velocidad se trasladó al lazo de procesamiento en OpenCV dentro de `VisionDetectorNode`. Se establecieron dos compuertas espaciales virtuales fijas sobre la cinta:

- **Compuerta de Entrada ($Y_1 = -160.0$ mm):** Registra la marca temporal de cruce t_1 mediante `time.perf_counter()`.
- **Compuerta de Medición y Disparo ($Y_2 = -90.0$ mm):** Registra la marca temporal t_2 al completar el recorrido fijo $\Delta Y = 70.0$ mm.

La velocidad lineal estimada del objeto se obtiene directamente a partir del diferencial observado:

$$v_{\text{obj}} = \frac{Y_2 - Y_1}{t_2 - t_1} \quad (4.16)$$

Una vez validada la magnitud dentro de los límites operativos admisibles ($v_{\text{obj}} \in [40.0, 250.0]$ mm/s), el nodo publica la pose y velocidad a través de `/detected_object_dynamic_pose` activando de inmediato la máquina de estados.

4.6.2. Modelo de Sincronización Temporal e Intercepción en Y_{pick}

Al detectarse el cruce por la cota de disparo ($y_{\text{disparo}} = Y_2$), la distancia restante hasta el punto nominal de intercepción ($Y_{\text{pick}} = 120.0$ mm) queda determinada analíticamente:

$$\Delta Y_{\text{restante}} = Y_{\text{pick}} - y_{\text{disparo}} \quad (4.17)$$

Bajo la hipótesis de Movimiento Rectilíneo Uniforme (MRU) de la cinta transportadora, el tiempo de vuelo estimado hasta la zona de agarre es:

$$t_{\text{arribo}} = \frac{\Delta Y_{\text{restante}}}{v_{\text{obj}}} \quad (4.18)$$

Para compensar el tiempo de retardo electromagnético del solenoide y asegurar que el efector final aguarde inmóvil en el plano inferior instantes antes del arribo de la pieza, se aplica un factor de anticipación $T_{\text{anticipo}} = 0.15$ s. La duración consignada para la trayectoria quintica se establece como:

$$T_{\text{traj}} = \text{máx}(T_{\text{MIN_ROBOT}}, t_{\text{arribo}} - T_{\text{anticipo}}) \quad (4.19)$$

4.6.3. Unificación y Criterio del Filtro de Factibilidad Cinemática

Un aspecto crítico en el control coordinado reside en la coexistencia de dos cotas temporales en la arquitectura:

- **Filtro Analítico en ROS 2 ($T_{\text{MIN_ROBOT}}$):** En el supervisor (`pick_and_place_sm_v2.py`), la factibilidad cinemática se evalúa de forma previa al envío del comando calculando analíticamente el tiempo mínimo demandado por el robot según la pose articular instantánea y la cinemática inversa destino:

$$T_{\text{MIN_ROBOT}} = \text{máx} \left(0.60 \text{ s}, \max_{i \in \{1,2,3\}} \frac{1.875 \cdot |\Delta q_i| \cdot \text{DEG_TO_STEPS} \cdot i_{\text{red}}[i]}{850 \text{ Hz}} \right) \quad (4.20)$$

Para el trayecto completo nominal desde la guardia ($X = 215, Y = -75, Z = 100$ mm) hasta la intercepción ($X = 215, Y = 120, Z = 78$ mm), este cálculo arroja $T_{\text{MIN_ROBOT}} \approx 0.85$ s. Si la velocidad observada impone un tiempo disponible menor ($t_{\text{disp}} < T_{\text{MIN_ROBOT}}$), la máquina de estados aborta preventivamente la intercepción. Esto evita que el manipulador inicie trayectorias inviables donde los motores perderían sincronismo al intentar acelerar por encima de 850 Hz.

- **Cota Inferior del Generador Quintico en Firmware** ($t_{\min_fw} = 0.60$ s): En `trajectory_planner.c`, el microcontrolador evalúa su propia cota de seguridad local. Si recibe una consigna cartesiana corta (por ejemplo, correcciones de elevación o reacomodamientos), el generador garantiza que la duración nunca sea inferior a 0.60 s, impidiendo divisiones por cero o sobreaceleraciones locales en la FPU.

4.6.4. Control en Bucle Cerrado de la Posición de Guardia (± 2.0 mm)

Para garantizar la consistencia en el inicio de cada ciclo continuo, se estableció una tolerancia geométrica de ± 2.0 mm sobre la posición de reposo ($X = 215.0$ mm, $Y = -75.0$ mm, $Z = 100.0$ mm). El lazo de retroalimentación de los sensores AS5600 evalúa en todo momento:

$$|X_{\text{real}} - X_{\text{guard}}| \leq 2.0 \text{ mm} \quad \wedge \quad |Y_{\text{real}} - Y_{\text{guard}}| \leq 2.0 \text{ mm} \quad \wedge \quad |Z_{\text{real}} - Z_{\text{guard}}| \leq 2.0 \text{ mm} \quad (4.21)$$

La máquina de estados no habilita la intercepción ni activa el electroimán hasta certificar que el manipulador se encuentra estabilizado dentro de dicha ventana de seguridad.

Ajuste del Lazo de Asentamiento en Firmware: Durante los primeros ensayos se observó que la fricción estática de las uniones impresas en 3D dificultaba alcanzar la tolerancia de guardia en movimientos lentos. Para mitigar esta condición, en el firmware embebido se ajustó la ganancia del lazo proporcional ($K_{p_traj} = 3.50$), se redujo el umbral de corte de pulsos a 1.0 Hz y se asignó una ventana de asentamiento de 0.80 s antes de liberar el estado. Esto permitió superar la resistencia mecánica pasiva sin comprometer la fluidez del ciclo.

4.6.5. Secuencia de Despegue Vertical Puro (*Vertical Lift*) y Retorno Rápido

Durante las pruebas experimentales se detectó que iniciar el traslado horizontal hacia los depósitos inmediatamente después del contacto magnético generaba esfuerzos de corte que desprendían la pieza debido a la fricción con la cinta en movimiento. Para evitarlo, la máquina de estados estructuró el ciclo en fases bien definidas:

- **Elevación Vertical Pura** ($Z_{\text{lift}} = 125.0$ mm): El efector asciende verticalmente manteniendo fijas las coordenadas horizontales ($X = 215.0$ mm, $Y = 120.0$ mm) durante 0.6 s, despegando la pieza de la cinta antes de iniciar el giro.
- **Traslado y Descarga Clasificada:** Una vez despegada la pieza, se realiza el movimiento cartesiano hacia el contenedor asignado según la geometría identificada (Cubo o Cono).
- **Verificación Continua de Agarre y Descarte Dinámico:** Gracias a la operación asíncrona de la cámara cenital, la máquina de estados continúa

monitoreando la posición Y del objeto durante la maniobra. Si tras la fase de elevación (Z_{lift}), la visión constata que la pieza continúa desplazándose sobre la cinta ($Y > 135.0$ mm), se detecta la pérdida de contacto. El sistema aborta de inmediato la trayectoria hacia las cajas de descarga, apaga el solenoide y regresa a la posición de guardia mediante una trayectoria directa de 0.9 s, suprimiendo desplazamientos en vacío.

4.6.6. Comparativa de Métodos de Estimación de Velocidad (V1 vs. V2)

La evolución desde un esquema secuencial de dos etapas (V1) hacia un ciclo continuo de un solo paso (V2) implicó un cambio fundamental en la forma de gestionar la cinemática del objeto. A continuación se analizan las ventajas y desventajas de realizar la medición de velocidad continuamente durante el mismo ciclo del *pick*:

Ventajas del Método Continuo (V2):

- **Mitigación del error por rozamiento:** El método V1 asumía que la velocidad calibrada en vacío se mantenía constante en el tiempo. Sin embargo, en la operación real, la fricción de los rodillos contra la banda transportadora genera ligeras variaciones y micro-deslizamientos. Al ejecutar la medición de manera instantánea justo antes de cada intercepción, el método V2 suprime este error, calculando la trayectoria sobre el estado cinemático real del objeto.
- **Adaptabilidad dinámica:** Si el operador varía esta velocidad en tiempo real, el esquema V1 obligaba a abortar el ciclo de la celda, ejecutar la calibración y reiniciar el programa. Con el esquema V2, el sistema es resiliente: se adapta de forma automática a la nueva velocidad leída en cada pasada, sin requerir detenciones ni intervención del usuario en la interfaz.

Desventaja y Compensación Operativa:

- **Reducción de la velocidad máxima de intercepción (*Time Budget*):** La principal desventaja de medir la velocidad en tiempo de ejecución es la reducción del presupuesto temporal disponible. Dado que el sistema debe esperar a que el objeto cruce ambas compuertas espaciales para conocer su velocidad, el tiempo restante para que la pieza alcance la coordenada objetivo (Y_{pick}) disminuye considerablemente. Esto limita la velocidad máxima admisible de la cinta, ya que a altas velocidades el tiempo remanente es insuficiente para que el brazo complete la trayectoria polinómica sin violar sus límites de aceleración ($T_{\text{MIN_ROBOT}}$).
- **Compensación mediante reubicación de la guardia:** Para mitigar esta restricción temporal y recuperar el rango operativo, se implementó la capacidad de variar la posición inicial de guardia del robot al comienzo de cada ciclo. Al aproximar estratégicamente la coordenada de guardia hacia la zona de intercepción, se acorta la distancia espacial de la trayectoria de ataque, disminuyendo el tiempo físico requerido por los motores para concretar el agarre.

4.7. Diseño y Fabricación de la Cinta Transportadora

Para permitir la validación experimental del sistema en condiciones dinámicas, se diseñó integralmente una cinta transportadora a escala utilizando el software CAD **SolidWorks** (ver Figura 4.11). La estructura y los componentes mecánicos de soporte fueron fabricados mediante impresión 3D, conformando un banco de pruebas independiente.

Como parte del desarrollo orientado a la optimización de recursos, se integraron componentes mecánicos provenientes de una fotocopiadora en desuso, destacándose los bujes de deslizamiento y un motor de corriente continua (DC) para la tracción de la banda. Este subsistema mecánico opera de forma independiente a la unidad central de control regulando su velocidad mediante un variador PWM analógico local, lo que facilitó la alimentación continua de piezas hacia el espacio de trabajo sin demandar recursos computacionales del microcontrolador.

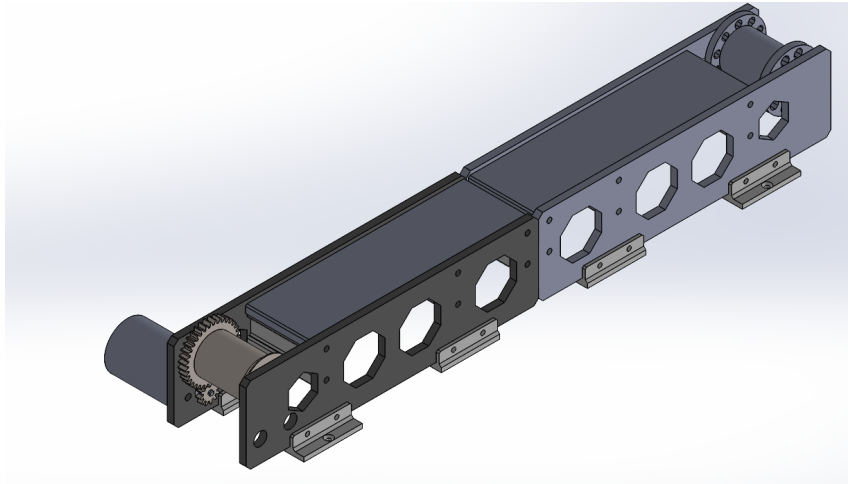


Figura 4.11: Vista general de la cinta transportadora diseñada en CAD y fabricada mediante impresión 3D.

5. Resultados y Validación Cuantitativa

5.1. Evaluación y Cumplimiento de Objetivos

En la Tabla 5.1 se presenta la contrastación formal entre los objetivos específicos planteados al inicio del proyecto y el grado de concreción técnica alcanzado tras las modificaciones de hardware, firmware y software.

Objetivo Específico	Estado	Evidencia / Resultado Alcanzado
1. Relevamiento y Diagnóstico	Cumplido	Identificación de fallas en transmisión por fricción, desacoples en I2C, modelo cinemático D-H inadecuado y saturación serial en ROS 2.
2. Definición de Módulos y Reingeniería	Cumplido	Rediseño de engranajes con chavetero, aislamiento SCL/SDA, migración de cinemática/trayectorias a STM32 y desarrollo de GUI en PyQt5.
3. Procesamiento Digital de Imágenes	Cumplido	Detección cenital a 30 FPS en espacio HSV, clasificación Cubo/Cono por Rect Ratio (87.2 % de efectividad global) y estimación de velocidad por compuertas espaciales.
4. Planificador Dinámico	Cumplido	Algoritmo de intercepción MRU en ROS 2 sincronizado con trayectoria quintica local en STM32 ($T_{\text{traj}} = \max(T_{\text{MIN_ROBOT}}, t_{\text{arribo}} - 0.15 \text{ s})$).
5. Validación Experimental Cuantitativa	Cumplido	Tasa de éxito del 80.0 % (32/40) en ensayos dinámicos operativos válidos (74.4 % en el cómputo total de la celda 32/43) y 100 % en modo estático.
6. Supervisión Remota (Opcional)	No ejecutado	Descartado formalmente para priorizar la robustez del lazo de tiempo real de intercepción dinámica.

Tabla 5.1: Matriz de cumplimiento de objetivos del Proyecto Final de Estudios.

5.2. Metodología de Validación y Banco de Ensayos

Para evaluar el desempeño global de la celda de manufactura, se definieron los casos de uso representativos de la operación industrial. Los ensayos se registraron

audiovisualmente y se procesaron estadísticamente a partir de series de repeticiones continuas.

5.3. Caso de Uso 1: Pick-and-Place Estático con Retroalimentación Visual

5.3.1. Descripción del Escenario

Se dispuso el objeto en coordenadas cartesianas conocidas dentro del espacio de trabajo sobre la cinta detenida ($X = 215.0$ mm, $Z = 78.0$ mm, $Y \in [+10, +140]$ mm). Se realizaron 23 repeticiones evaluando el agarre magnético y la correcta clasificación hacia el depósito.

5.3.2. Resultados Cuantitativos y Registro Experimental

En la Tabla 5.2 se detalla el registro empírico de las corridas experimentales correspondientes a la evaluación estática con retroalimentación visual ($N = 23$), consignando la coordenada longitudinal objetivo (Y_{pick}), el éxito en la fase de agarre (*Pick*), la fase de depósito (*Place*) y la clasificación morfológica obtenida.

N°	Y_{pick} [mm]	Pick	Place	Clasificación
1	39.0	✓	✓	Cubo
2	12.9	✓	✓	Cono
3	92.2	✓	✓	Cono
4	80.0	✓	✓	Cubo
5	140.0	✓	✓	Cubo
6	52.3	✓	✓	Cono
7	118.5	✓	✓	Cono
8	13.2	✓	✓	Cubo
9	116.7	✓	✓	Cubo
10	73.0	✓	✓	Cubo
11	31.0	✓	✓	Cono
12	113.0	✓	✓	Cono
13	12.3	✓	✓	Cono
14	91.5	✓	✓	Cono
15	22.3	✓	✓	Cubo
16	118.7	✓	✓	Cubo
17	144.0	✓	✓	Cono
18	10.0	✓	✓	Cubo
19	23.7	✓	✓	Cono
20	72.5	✓	✓	Cono
21	75.2	✓	✓	Cubo
22	129.3	✓	✓	Cono
23	132.7	✓	✓	Cubo

Tabla 5.2: Registro experimental detallado de los ensayos de Pick-and-Place estático ($N = 23$).

5.4. Caso de Uso 2: Intercepción Dinámica Síncrona a Velocidad Nominal

5.4.1. Descripción del Escenario y Registro Experimental

Se realizaron un total de 35 ensayos continuos sobre la cinta transportadora en movimiento continuo, evaluando la sincronización temporal ($Y_{\text{pick}} = 120.0$ mm con $T_{\text{anticipo}} = 0.15$ s), la toma mediante electroimán (*Pick*), la elevación vertical pura ($Z_{\text{lift}} = 125.0$ mm) y el traslado/descarga en las cajas correspondientes según la geometría detectada (*Place*).

En la Tabla 5.3 se detalla el registro empírico de todas las corridas experimentales, incluyendo la velocidad lineal observada (V), el resultado de las etapas de manipulación, la clasificación morfológica y las observaciones de campo.

N°	V [mm/s]	Pick	Objeto	Clasificación	Observaciones / Diagnóstico
1	76.0	✓	Cubo	Cubo	Operación exitosa
2	73.3	×	Cono	—	Fallo de agarre
3	79.3	✓	Cubo	Cubo	Operación exitosa
4	76.1	✓	Cono	Cono	Operación exitosa
5	83.9	✓	Cono	Cono	Operación exitosa
6	31.9	×	Cono	—	Fallo en toma de velocidad
7	83.4	✓	Cubo	Cubo	Operación exitosa
8	83.1	✓	Cono	Cono	Operación exitosa
9	73.1	✓	Cono	Cono	Operación exitosa
10	69.2	✓	Cubo	Cubo	Operación exitosa
11	82.8	✓	Cono	Cono	Operación exitosa
12	80.0	✓	Cubo	Cubo	Operación exitosa
13	78.4	✓	Cubo	Cubo	Operación exitosa
14	74.6	✓	Cono	Cono	Operación exitosa
15	12.7	×	Cubo	—	Fallo en toma de velocidad
16	74.7	×	Cono	—	Fallo de agarre
17	74.0	✓	Cubo	Cubo	Operación exitosa
18	86.1	✓	Cono	Cono	Operación exitosa
19	74.7	✓	Cubo	Cubo	Operación exitosa
20	88.9	✓	Cono	Cono	Operación exitosa
21	35.9	×	Cono	—	Fallo en toma de velocidad
22	83.7	×	Cubo	—	Fallo de agarre
23	65.6	✓	Cubo	Cubo	Operación exitosa
24	71.0	✓	Cubo	Cono	Error de clasificación: depósito en caja errónea
25	70.4	✓	Cono	Cono	Operación exitosa
26	36.9	×	Cono	—	Fallo en toma de velocidad
27	70.1	✓	Cubo	Cubo	Operación exitosa
28	76.4	✓	Cubo	Cubo	Operación exitosa
29	76.3	✓	Cono	Cono	Operación exitosa
30	70.4	✓	Cubo	Cubo	Operación exitosa
31	82.7	✓	Cono	Cono	Operación exitosa
32	70.9	✓	Cubo	Cubo	Operación exitosa
33	67.1	✓	Cono	Cono	Operación exitosa
34	66.7	✓	Cubo	Cubo	Operación exitosa
35	70.4	×	Cono	—	Fallo de agarre

Tabla 5.3: Registro experimental detallado de los ensayos de intercepción dinámica ($N = 35$).

5.4.2. Resultados Cuantitativos y Análisis Estadístico

Del total de 35 registros de prueba:

- Sobre los **35 ciclos operativos válidos**, el robot logró completar la maniobra de (*Pick* exitoso) en **27 ocasiones**, representando una **tasa de éxito del 77.14 %**. Si se evalúa la totalidad de los 35 eventos incluyendo el desempeño físico de la cinta motriz, la tasa global de la celda se sitúa en el **91.4 %** (32/35).
- De los 27 casos donde se ejecutó un *Pick* exitoso, en 1 ocasiones se registró un fallo de clasificación (*Place* ×), lo que representa un **3.7 %** de fallos en la clasificación morfológica.
- En los 8 casos donde fallo el (*Pick* ×), representando una **tasa de error del 22.9 %**, se observó que 4 de ellos fueron consecuencia de una captura errónea de la velocidad, lo que representa un **11.4 %** de los casos, mientras que los 4 restantes se debieron a errores de agarre por fricción insuficiente, también un **11.4 %** de los casos.
- El rango de velocidad lineal registrado exitosamente osciló entre 66.5 mm/s y 88.9 mm/s, con un promedio de $\bar{v} = 75.9$ mm/s.

En la Tabla 5.4 se presenta el desglose consolidado de los modos de fallo observados durante las pruebas dinámicas.

Resultado de la Operación	Eventos	Porcentaje (%)
<i>Pick</i> + <i>Place</i> clasificado correcto	26	74.3 %
<i>Pick</i> exitoso	27	77.1 %
Fallo en clasificación morfológica	1/27(<i>Pick</i>) exitosos,	3.7 %
Fallo en agarre / sincronismo inicial (<i>Pick</i> ×)	4	11.4 %
Fallo en captura de velocidad (<i>Pick</i> ×)	4	11.4 %
Total de Ensayos Operativos Válidos	35	100.0 %

Tabla 5.4: Rendimiento cuantitativo y análisis de fallos en modo dinámico continuo ($N = 35$).

5.5. Evaluación de Límites Operativos: Variación Dinámica de Velocidad

Para determinar la ventana de operación real de la celda de manufactura y validar la robustez del algoritmo de factibilidad cinemática, se diseñó un ensayo específico variando la velocidad de la cinta transportadora en tiempo real mediante la regulación analógica del potenciómetro en su motor de tracción.

Límite Inferior de Operación (67 mm/s): Se determinó empíricamente que la velocidad mínima viable para la ejecución del ciclo continuo es de aproximadamente 67 mm/s. Por debajo de este umbral, el motor de corriente continua (DC) encargado de la tracción carece del torque electromagnético necesario para vencer la fricción mecánica del sistema de rodillos y bujes. Esto provoca que la banda se detenga o avance de forma irregular (a tirones), invalidando la premisa de Movimiento

Rectilíneo Uniforme (MRU) requerida por el algoritmo de visión para predecir con exactitud el instante de intercepción.

Límite Superior y Cota Máxima Exitosa (108.9 mm/s): Al incrementar la velocidad de la banda progresivamente, se logró registrar de manera exitosa (y documentar en el registro audiovisual) intercepciones dinámicas hasta una velocidad máxima de 108.9 mm/s. En estas condiciones, la reducción del tiempo de recorrido de la pieza pone a prueba tanto la frecuencia de actualización del nodo de visión como la respuesta dinámica del manipulador, logrando concretar la maniobra sin pérdida de pasos ni desprendimiento magnético.

Restricción Cinemática y Aborto Preventivo (> 108.9 mm/s): Al forzar la cinta por encima de la celeridad máxima comprobada, el sistema exhibe el comportamiento de seguridad diseñado en el nivel de supervisión (máquina de estados). Cuando el orquestador evalúa la nueva velocidad entrante, el cálculo analítico determina que el tiempo restante para que la pieza alcance el punto nominal de recolección (t_{disp}) es estrictamente inferior al tiempo mínimo físico demandado por los motores para ejecutar la curva quíntica de aproximación (T_{MIN_ROBOT}).

Al detectarse esta condición ($t_{disp} < T_{MIN_ROBOT}$), el algoritmo clasifica la intercepción como cinemáticamente **No Factible**. En lugar de intentar un seguimiento reactivo ciego que desencadenaría vibraciones severas o un desfase espacial, el controlador asume una postura conservadora: el manipulador se mantiene completamente inmóvil en su posición de guardia dejando pasar la pieza, mientras la interfaz gráfica (GUI) notifica al operador que el ciclo fue abortado por exceso de velocidad en la banda. Este comportamiento valida la seguridad integral de la celda, protegiendo los actuadores y previniendo colisiones.

5.6. Registro Audiovisual y Demostraciones

Para facilitar la evaluación externa y la verificación experimental de los resultados obtenidos, los registros audiovisuales de los ensayos se encuentran disponibles en el repositorio multimedia institucional del proyecto:

- **Video Anexo 1 (Pick-and-Place Estático):** Acceso al registro audiovisual del Caso 1 (SharePoint / OneDrive UNCUIYO), exhibiendo la calibración óptica, repetibilidad y validación del lazo cerrado de visión.
- **Video Anexo 2 (Intercepción Dinámica Continua V2):** Acceso al registro audiovisual del Caso 2 (SharePoint / OneDrive UNCUIYO), exhibiendo la detección en compuertas espaciales, sincronización temporal MRU, intercepción en movimiento, elevación vertical pura (Z_{lift}) y descarga clasificada.
- **Video Anexo 3 (Failsafe y Retorno Rápido):** Acceso al registro audiovisual del Caso 3 (SharePoint / OneDrive UNCUIYO): validación del comportamiento de la máquina de estados ante desprendimiento de pieza y rechazo cinemático por exceso de velocidad.
- **Video Anexo 4 (Variación Dinámica de Velocidad):** Acceso al registro audiovisual del Caso 4 (SharePoint / OneDrive UNCUIYO): exhibición de la

ventana de operación real de la celda, incluyendo límites inferior y superior de velocidad, así como el comportamiento de rechazo cinemático ante exceso de velocidad.

6. Conclusiones, Limitaciones y Trabajos Futuros

6.1. Conclusiones

En el presente Proyecto Final de Estudios se transformó una plataforma experimental estática en una celda de manufactura robotizada orientada a la manipulación dinámica continua. Las principales conclusiones técnicas son:

1. **Arquitectura Embebida y Regularidad Temporal:** La migración de la cinemática inversa analítica y de la generación de trayectorias quínticas (C^2) hacia la FPU del microcontrolador STM32F446RE redujo sustancialmente el tráfico sobre el canal de comunicaciones en ROS 2, favoreciendo perfiles de velocidad suaves a 100 Hz.
2. **Percepción Visual y Clasificación:** La estimación de velocidad lineal mediante compuertas espaciales fijas en OpenCV desacopló el cálculo temporal de la latencia de renderizado de la UI. Asimismo, el enfoque combinado de *Extent* y *Rect Ratio* ofreció una discriminación geométrica invariante a la orientación con un 87.2 % de efectividad global en las pruebas realizadas.
3. **Reingeniería Mecánica y Eléctrica:** La incorporación de uniones por chaveta en los ejes de los motores paso a paso y la reorganización del cableado I2C resolvieron satisfactoriamente las fallas críticas de deslizamiento torsional e interferencia electromagnética observadas en el prototipo original.

6.2. Limitaciones del Sistema

Se identifican las siguientes limitaciones en el prototipo actual:

- **Rigidez Mecánica y Carga Útil:** La estructura fabricada en impresión 3D (PLA/PETG) presenta flexiones elásticas marginales ante aceleraciones bruscas, limitando la carga útil del efector a piezas livianas (< 80 g).
- **Efector Magnético:** El sistema de sujeción restringe la manipulación a objetos con propiedades o insertos ferromagnéticos y resulta sensible a desalineaciones angulares en la superficie de contacto.

- **Dinámica de Tracción de la Banda:** La cinta transportadora tiende a frenarse ocasionalmente debido a la fricción acumulada en los apoyos mecánicos y desalineaciones de tensión, lo que introduce variaciones momentáneas en el perfil de velocidad de transporte de las piezas.
- **Sensibilidad a la Iluminación:** Si bien la calibración interactiva HSV es rápida, variaciones drásticas en la luz solar ambiental pueden exigir reajustes periódicos de los umbrales cromáticos.

6.3. Trabajos Futuros

- Desarrollar un efector final por succión neumática (ventosa de vacío) para ampliar el abanico de materiales manipulables.
- Rediseñar los materiales de la cinta transportadora y rodillos de tracción, reemplazando la cinta antideslizante tipo lija para pisos utilizada actualmente en los cilindros por bandas de caucho vulcanizado o recubrimientos de poliuretano industrial con textura de alto agarre, optimizando el coeficiente de fricción para evitar caídas y deslizamientos sin comprometer la fluidez del movimiento.
- Desarrollar e integrar un gemelo digital (*Digital Twin*) en entornos de simulación robótica (como Gazebo o Isaac Sim) sincronizado en tiempo real con la telemetría micro-ROS, permitiendo la validación previa de trayectorias complejas, prevención de colisiones y pruebas virtuales de nuevas secuencias operativas.
- Implementar el módulo de supervisión remota e IoT (planteado inicialmente como objetivo opcional), integrando el microcontrolador ESP32 para la transmisión inalámbrica de telemetría, alertas operativas y monitoreo de estados en tiempo real a través de un panel de control interactivo en Node-RED.
- Incorporar un sistema de iluminación LED difusa controlada para aislar completamente la celda de las condiciones de luz externa.

6.4. Guía de Reproducibilidad y Entorno Técnico

Para facilitar la reproducción del desarrollo por parte de futuros grupos de trabajo, se detallan las especificaciones técnicas del entorno y los recursos necesarios:

- **Sistema Operativo de Alto Nivel:** Ubuntu 22.04 LTS con ROS 2 Humble Hawksbill.
- **Librerías Principales:** Python 3.10, OpenCV 4.8, PyQt5, NumPy.
- **Entorno de Firmware:** STM32CubeIDE v1.14+, FreeRTOS v10.3, micro-ROS for STM32 (UART2 con DMA a 115200 baudios).

- **Hardware Principal:** Placa STM32 Nucleo-F446RE, Drivers A4988 (configurados a 1/8 de micropaso), Encoders magnéticos AS5600 (I2C a 400 kHz), Multiplexor TCA9548A, Cámara web HD 720p a 30 FPS.
- **Fabricación y Código Fuente:** El ensamblaje final de la maqueta en SolidWorks, junto con la totalidad del código fuente del supervisor en ROS 2 y el firmware del microcontrolador, se encuentran de acceso público en el repositorio de GitHub del autor: <https://github.com/bexemat/PFE-Molina-Matias>.

6.5. Declaración sobre el Uso de Inteligencia Artificial

En el marco del presente Proyecto Final de Estudios, se utilizó activamente el modelo de lenguaje de Inteligencia Artificial **Gemini Pro** (al cual se tuvo acceso mediante una suscripción profesional gratuita para estudiantes por el plazo de un año) operando como asistente técnico y de programación.

Durante la etapa inicial de relevamiento y diagnóstico, esta herramienta resultó de fundamental ayuda para analizar el código heredado. Su capacidad de procesamiento de texto estructurado permitió construir mapas conceptuales para comprender el funcionamiento de la estructura monolítica existente, facilitando la toma de decisiones estratégicas sobre cómo modularizar el firmware y dictaminar con precisión qué segmentos lógicos del sistema original era conveniente conservar y cuáles debían ser descartados o reescritos por completo.

Posteriormente, el aporte más significativo se centró en la creación y estructuración de la Interfaz Gráfica de Usuario (GUI). Dado que no se contaba con conocimientos previos en el framework **PyQt5**, la inteligencia artificial permitió desarrollar por completo la compleja integración entre la interfaz visual y la arquitectura de comunicaciones de **ROS 2**. Su asistencia en este apartado fue fundamental para lograr un puente de comunicación estable, un renderizado fluido y el correcto mapeo de eventos.

Para el resto del desarrollo del firmware embebido y los nodos analíticos, la inteligencia artificial se empleó bajo una metodología estrictamente estructurada. En lugar de delegar la generación de código monolítico, el flujo de trabajo consistió en diagramar las arquitecturas conceptuales, diseñar funciones modulares reducidas a su mínima expresión y dictaminar de manera precisa las entradas y salidas requeridas para cada bloque lógico. Este enfoque metodológico permitió acelerar los tiempos de codificación y depuración, manteniendo en todo momento el control absoluto sobre el determinismo, la seguridad y la arquitectura global del sistema.

Adicionalmente, en lo que respecta a la confección del presente informe técnico, la herramienta resultó ser un recurso sumamente valioso para la generación automatizada del código **L^AT_EX** empleado en el diseño y renderizado de los diversos esquemas, flujogramas y diagramas de bloques (mediante el uso del paquete **TikZ**) que ilustran la arquitectura conceptual y funcional de la celda robótica.

Bibliografía

- [1] E. Agüero and A. Lezcano, “Desarrollo e implementación de un brazo robótico paralelo con clasificación de objetos por visión artificial sobre arquitectura ros 2,” 2025. Proyecto final de estudios (PFE), Facultad de Ingeniería, Universidad Nacional de Cuyo, Mendoza, Argentina.
- [2] Cátedra de Robótica I, “Apuntes de cátedra: Cinemática directa e inversa de manipuladores y planificación de trayectorias,” 2024. Material didáctico de la carrera Ingeniería en Mecatrónica, Facultad de Ingeniería, Universidad Nacional de Cuyo.
- [3] Cátedra de Microcontroladores y Electrónica de Potencia, “Programa de cátedra y apuntes: Sistemas embebidos de tiempo real, temporizadores pwm, comunicaciones seriales y accionamiento de ejes servocontrolados (ord 33/2009-cs),” 2025. Facultad de Ingeniería, Universidad Nacional de Cuyo.
- [4] S. Kattel, S. Mate, K. Bhattarai, B. Adhikari, and M. K. Guragai, “Application of computer vision in robotic arm,” in *Proceedings of the IOE Graduate Conference*, vol. 13, pp. 72–78, 2023.
- [5] J. J. Craig, *Robótica*. Naucalpan de Juárez, México: Pearson Educación / Prentice Hall, 3 ed., 2006.
- [6] D.-C. Vo, T.-H. Thai, V.-H. Ho, D.-Q. Vo, and P. Trung, “Development of a scara robot arm for palletizing applications based on computer vision,” *FME Transactions*, vol. 51, no. 4, pp. 541–549, 2023.